

队伍编号	260031
题号	C

中老年人群高血脂症的风险预警及干预方案优化

摘要

本文针对中老年人群高血脂症的风险预警及干预方案优化问题，融合中医体质学、血常规体检指标及日常活动能力评分，构建了多维度数学模型。

针对问题 1，构建“Pearson/Spearman 相关 + 组间差异检验 + 逐步 Logistic 回归”的三层筛选体系，发现血脂指标（TC、TG、LDL-C、HDL-C）与尿酸对高血脂标签有显著预警能力，但与痰湿质积分的线性关联极弱（调整 $R^2 \approx 0$ ）。据此将关键指标明确区分为两类：高血脂风险识别指标（TC、TG、LDL-C、HDL-C、尿酸）与痰湿严重程度直接表征指标（痰湿质积分、主体质标签）。进一步引入随机森林模型弥补线性模型在交互效应上的不足。通过多元 Logistic 回归量化九种体质贡献度，各体质标准化系数均未达显著性水平；随机森林 Gini 重要性提示阴虚质、湿热质、血瘀质的非线性特征重要性排名靠前，提示体质间可能存在复杂的非线性交互效应。

针对问题 2，针对高血脂标签与血脂指标存在定义同源性的问题，将风险建模拆分为两层：第一层为现症识别模型（血脂指标复现诊断， $AUC \approx 0.978$ ），第二层为治未病预警模型（体质 + 活动 + 基础信息， $AUC \approx 0.597$ ）。该 AUC 反映非血脂指标对高血脂标签的解释力有限，预警模型旨在发病前识别高危人群，而非复现诊断。综合风险评分采用熵权法与专家经验组合赋权，结合统计分位数与临床锚点确定三级阈值。5 折交叉验证与 ROC 分析量化模型区分能力，权重敏感性分析验证分层稳健性；通过条件概率识别核心特征组合。

针对问题 3，建立基于时间序列的离散动态规划模型，将 6 个月干预过程逐月模拟，实现调理分级随痰湿积分下降而动态自适应降级（“随症加减”），在保证干预效果的同时节约后期过度医疗成本。引入依从性衰减函数刻画老年患者耐受度差异，并开展敏感性分析。求解得到样本 ID 1、2、3 的月度干预轨迹及全体 278 例痰湿体质患者的最优方案，总结出“患者特征—最优方案”匹配规律。

关键词：高血脂症；中医体质；随机森林；风险分层；动态规划；干预优化

目录

1	问题重述与分析	1
1.1	问题背景	1
1.2	问题提出	1
1.3	问题分析	1
2	模型假设与符号说明	2
2.1	基本假设	2
2.2	符号说明	2
3	问题 1：关键指标筛选与体质贡献度	3
3.1	数据预处理	3
3.2	关键指标筛选方法	3
3.2.1	相关性分析与组间差异检验	3
3.2.2	逐步 Logistic 回归	4
3.3	九种体质贡献度分析	4
3.4	问题 1 结果与分析	5
3.4.1	相关性分析与组间差异检验结果	5
3.4.2	逐步回归筛选结果	7
3.4.3	体质贡献度分析结果	7
4	问题 2：三级风险预警模型	9
4.1	两层风险建模框架	9
4.2	综合风险评分构造	9
4.3	三级风险阈值确定	10
4.3.1	统计分位数法	10
4.3.2	临床锚定与阈值校准	10
4.4	三层判定规则	10
4.5	核心特征组合识别	11
4.6	模型验证：5 折分层交叉验证与 ROC 分析	11
4.7	权重敏感性分析	12
4.8	问题 2 结果与分析	12
4.8.1	风险分层结果	12
4.8.2	高风险人群画像	13
4.8.3	交叉验证与 ROC 结果	13
4.8.4	权重敏感性分析结果	14
4.8.5	问题 2 小结	15

5 问题 3：干预方案优化模型	16
5.1 模型建立与分析	16
5.1.1 决策变量与参数设定	16
5.1.2 约束条件	16
5.1.3 目标函数与依从性约束	17
5.2 求解方法：穷举法逐月模拟	18
5.3 问题 3 结果	19
5.3.1 样本 ID 1、2、3 月度干预轨迹	19
5.3.2 “患者特征—最优方案”匹配规律	20
5.3.3 依从性衰减敏感性分析	22
6 模型评价与改进	23
6.1 模型优点	23
6.2 模型缺点	23
6.3 模型改进方向	24
A 附录	27
A.1 软件及运行环境说明	27
A.2 主要计算机源程序	27
A.3 AI 使用情况说明	73

1 问题重述与分析

1.1 问题背景

人口老龄化加剧，中老年人群高血脂症发病率攀升，成为心脑血管疾病的重要危险因素。痰湿体质被视为血脂异常最常见的易感体质之一，其病机关键在于脾失健运、水湿内停、痰浊内生 [1, 2]。多项流行病学调查表明，痰湿体质在高脂血症患者中的占比可达 21%–28%，显著高于其他体质类型 [3, 4]。然而，不同地区的研究结果存在差异：成都地区的调查显示痰湿体质占原发性高脂血症患者的 24.7% [3]，深圳 2 型糖尿病合并血脂异常患者中痰湿体质占比为 21.4% [4]，提示地域、人群特征及判定标准均可能影响体质分布。此外，“治未病”思想强调通过体质辨识实现疾病前预警与个性化健康管理 [5, 6]，血清免疫球蛋白和补体角度的研究也为中医体质与动脉硬化的关联提供了实验依据 [20]。而日常活动能力直接影响气血运行与膏脂运化 [7]。目前临床筛查单一依赖血常规，缺乏对体质分型与活动能力的综合考量，难以实现“治未病”的精准防控。

1.2 问题提出

- 问题 1** 从血常规与活动量表中筛选能有效表征痰湿体质严重程度、且预警高血脂发病风险的关键指标；研究九种体质对发病风险的贡献度差异。
- 问题 2** 构建融合多维度特征的三级风险预警模型（低/中/高），明确阈值选取依据；识别痰湿体质高风险人群的核心特征组合。
- 问题 3** 针对痰湿体质患者，在中医调理原则、身体耐受度及经济成本约束下，构建 6 个月干预方案优化模型，给出样本 ID 1、2、3 的最优方案及“患者特征—最优方案”匹配规律。

1.3 问题分析

- **问题 1** 属于特征筛选与归因分析问题。可利用相关性分析与回归模型的显著性检验筛选关键指标；利用 Logistic 回归系数或优势比 (OR) 衡量体质贡献度。
- **问题 2** 属于风险分层与规则挖掘问题。核心是将连续风险概率离散化为三级，需兼顾统计分布与临床意义；核心特征组合可通过条件概率或决策树规则提取。
- **问题 3** 属于带约束的组合优化问题。决策空间极小（调理分级 × 活动强度 × 频率），可用穷举法精确求解；目标是降分最大化与成本可控的双重要求。

2 模型假设与符号说明

2.1 基本假设

1. 样本数据独立同分布，能代表目标中老年人群总体特征。
2. 中医调理分级严格按痰湿积分区间匹配，不可跨级选择。
3. 活动干预效果满足题目给出的线性叠加规则：基础强度效果与额外频率效果相互独立。
4. 每月按 4 周计，6 个月干预周期共 24 周。
5. 高龄（80-89 岁）及低活动能力（ $M < 40$ ）患者存在干预疲劳效应：每周超过 5 次的部分，实际有效频率依从性衰减（50%-80%），且设置安全上限 $f_{\max} = 7$ 次/周。该衰减基于老年医学“高龄及低活动能力患者运动耐受有限”的一般性共识设定，旨在反映定性规律，精确数值有待临床随访数据标定。

2.2 符号说明

表 1: 主要符号及其含义

符号	含义	单位/备注
x	中医调理分级	$x \in \{1, 2, 3\}$
y	活动干预强度	$y \in \{1, 2, 3\}$
f	每周训练频率	$f \in \{1, 2, \dots, 10\}$ （次/周）
S_0	初始痰湿积分	0 ~ 100 分
S_6	6 个月后痰湿积分	0 ~ 100 分
r	每月痰湿积分下降率	0 ~ 某正值
$C_T(x)$	调理分级 x 的月成本	{30, 80, 130} 元/月
$C_A(y)$	活动强度 y 的单次成本	{3, 5, 8} 元/次
M	活动量表总分 (ADL+IADL)	0 ~ 100 分
A	年龄组	1 ~ 5 (40~89 岁)
P	高血脂发病概率	0 ~ 1
R	综合风险评分	用于三级分层

3 问题 1：关键指标筛选与体质贡献度

3.1 数据预处理

原始数据为 Excel 工作簿，共 1000 例样本，包含 9 种体质积分、血常规指标（HDL-C、LDL-C、TG、TC、空腹血糖、血尿酸）、BMI、ADL/IADL 量表及年龄、性别等人口学特征。为保证建模质量，预处理分为以下四步：

(1) 数据整合与变量映射原始数据为 Excel 格式，共 1000 例样本、36 个字段。首先对列名进行标准化映射，将中文字段名转为英文简写以便后续建模。依据题目附表 1 的编码规则，将性别（0=女，1=男）、年龄组（1=40-49 岁至 5=80-89 岁）等分类变量转为数值型，确保类型一致。

(2) 方向性血脂异常风险项数构造高血脂风险具有明确的单向性：TC、TG、LDL-C 升高或 HDL-C 降低才构成风险。因此，本文采用方向性异常定义，仅将 $TC > 6.2$ 、 $TG > 1.7$ 、 $LDL-C > 3.1$ 、 $HDL-C < 1.04$ 计入血脂异常风险项数，记为 $N_{\text{lipid-risk}} \in \{0, 1, 2, 3, 4\}$ 。血尿酸正常范围存在性别差异（男 $208 \sim 428 \mu\text{mol/L}$ ，女 $155 \sim 357 \mu\text{mol/L}$ ），因此单独构造血尿酸异常标记 $U_{\text{abn}} \in \{0, 1\}$ ，不混入血脂异常项数，以保证指标定义的临床准确性。

(3) Z-score 标准化由于血常规指标量纲各异（如 TC 单位为 mmol/L ，BMI 为 kg/m^2 ），直接入模会导致大数量级变量压制小数量级变量。因此，对 TC、TG、LDL-C、HDL-C、血糖、尿酸、BMI、ADL、IADL 及活动量表总分进行标准化：

$$Z_{ij} = \frac{X_{ij} - \bar{X}_j}{S_j}, \quad (1)$$

其中 $\bar{X}_j$ 与 S_j 分别为第 j 个变量的样本均值与标准差。

3.2 关键指标筛选方法

3.2.1 相关性分析与组间差异检验

题目要求筛选的指标需同时满足两个功能：**表征痰湿体质严重程度与预警高血脂发病风险**。然而，如后文结果所示，本数据集中血常规指标与痰湿质积分的线性关联极弱，而对高血脂标签的预警能力显著。因此，本文不强行将两类功能归于同一组指标，而是从两个目标变量出发分别分析，建立三层筛选体系：

(1) Pearson 与 Spearman 相关分析针对痰湿质积分（连续变量），分别计算 Pearson 积矩相关系数（捕捉线性关系）与 Spearman 秩相关系数（捕捉单调非线性关系）。针对高血脂二分类标签（0/1），计算各指标与标签的 Pearson 相关系数（等价于点二列相关系数）。

(2) 组间差异检验（Mann-Whitney U 检验）由于痰湿质积分在样例数据中与血常规指标的线性相关性极弱（见后文结果），单一相关分析可能遗漏非线性关联。为此，引入基于体质标签的组间差异检验：将样本按主体质是否为痰湿质（体质标签 = 5）分为两组，对各血常规及活动量表指标进行 Mann-Whitney U 检验。Mann-Whitney U 检

验作为最强的非参数检验方法之一，不假设总体分布形式，对异常值具有鲁棒性，广泛应用于医学和气候学领域的极端事件检测 [8, 9]。效应量采用秩次相关系数 $r = Z/\sqrt{N}$ 衡量。

(3) **多元线性回归（控制混杂因素）** 进一步以痰湿质积分为因变量，10 项血常规及活动量表指标为自变量，建立多元线性回归模型，通过标准化系数（Beta 系数）比较各指标在控制其他变量后的独立贡献。

3.2.2 逐步 Logistic 回归

相关性分析仅能揭示变量间的线性关联，无法排除混杂因素，因此进一步建立 Logistic 回归模型，通过控制其他变量来识别独立预警指标。在多元 Logistic 回归分析中，自变量筛选及逐步回归方法的选择直接影响模型稳定性与解释力，已有研究 [10] 表明应综合采用向前选择、向后剔除及双向逐步策略，避免单一方向选择遗漏重要变量，同时若将多分类变量当作连续性变量处理会损失信息，甚至漏掉重要危险因素。设 $Y \in \{0, 1\}$ 为高血脂标签， $\mathbf{X} = (X_1, X_2, \dots, X_k)^T$ 为候选指标向量，Logistic 模型形式为：

$$\ln \frac{P(Y = 1|\mathbf{X})}{1 - P(Y = 1|\mathbf{X})} = \beta_0 + \sum_{i=1}^k \beta_i X_i, \quad (2)$$

其中左侧为事件发生比（Odds）的对数，称为 logit 函数； β_i 表示在控制其他变量后， X_i 每增加一个单位对 logit 的边际贡献。

为从高维候选指标中自动筛选最优子集，采用**前向选择（Forward Selection）**与**后向剔除（Backward Elimination）**相结合的逐步策略：

1. **前向选择**：从未选变量中找到 $p\text{-value} < \alpha_{in} = 0.05$ 的变量，加入模型；
2. **后向剔除**：对已选变量进行 Wald 检验，若某变量 $p\text{-value} > \alpha_{out} = 0.10$ ，则将其剔除；
3. 重复上述两步，直至没有变量可进也没有变量可出，算法收敛。

该策略相比全子集遍历大幅降低了计算量，同时通过双向校验减少了单一方向选择可能遗漏的重要变量。最终保留的变量即为在统计学意义上能够独立预警高血脂的关键指标。

3.3 九种体质贡献度分析

中医体质学将人体划分为九种基本类型，不同体质与疾病易感性存在差异，痰湿体质被认为是血脂异常最常见的易感体质之一，其脂代谢异常的病机关键在于脾失健运、痰浊内生 [1]，且肠道菌群角度的研究也为湿性体质人群脂代谢异常提供了微生物学机制解释 [7]。为量化各体质对高血脂发病风险的独立贡献，将 9 种体质积分同时纳入多

元 Logistic 回归。由于体质积分间可能存在相关性（如痰湿质与湿热质常相伴出现），直接比较原始系数会因量纲差异而产生偏误。因此，先对各体质积分进行 Z-score 标准化：

$$X_i^* = \frac{X_i - \bar{X}_i}{\sigma_i}, \quad i = 1, 2, \dots, 9, \quad (3)$$

标准化后，各变量均值为 0、标准差为 1，回归系数 β_i^* 可直接比较绝对值大小。

模型形式为：

$$\text{logit}(P) = \beta_0 + \sum_{i=1}^9 \beta_i^* X_i^*. \quad (4)$$

在解释贡献度时，综合考察两项指标：

- **标准化回归系数 $|\beta_i^*|$** ：反映该体质积分每增加 1 个标准差，logit 的变化幅度；
- **优势比 (OR)** $\text{OR}_i = \exp(\beta_i^*)$ ：反映该体质积分每增加 1 个标准差，患病优势 (Odds) 的倍数变化。OR>1 表示风险增加，OR<1 表示风险降低。

然而，传统 Logistic 回归作为全局线性模型，容易掩盖变量间的非线性交互效应。考虑到体质间的耦合效应（如痰湿质与湿热质常相伴出现），单一 Logistic 模型可能低估某些体质的实际影响。为此，本文进一步引入**随机森林 (Random Forest)** 模型，通过 Gini 不纯度减少量度量各体质积分的非线性特征重要性，作为 Logistic 回归的补充验证。随机森林对多重共线性和非线性交互具有天然鲁棒性，其特征重要性排名可揭示线性模型无法捕捉的复杂模式。现有文献对随机森林变量重要性度量进行了系统研究，涵盖最小深度、分裂频次、置换重要性及节点纯度等原理与性能分析 [11]，并在高维医学数据中展现了处理多重共线性和噪声变量的优势 [12]；特征重要性排序在医学数据挖掘中的稳定性亦得到验证 [13]，加权决策树优化进一步提升了重要性度量的精确度 [22]，为本文选择随机森林作为非线性补充验证工具提供了方法学支持。

3.4 问题 1 结果与分析

3.4.1 相关性分析与组间差异检验结果

表 2 展示了各指标与痰湿质积分及高血脂标签的相关系数。在**预警高血脂**方面，TG、TC 的点二列相关系数最高（分别为 0.467、0.439， $p \ll 0.001$ ），LDL-C、HDL-C 及尿酸也显示出中等强度关联，与临床认知一致，且已有研究表明痰湿体质高脂血症合并抑郁症患者的尿酸水平与痰湿体质及抑郁程度均呈正相关，提示尿酸可能是痰湿体质继发代谢紊乱的重要标志物之一 [21]。然而，在**表征痰湿体质严重程度**方面，所有血常规及活动量表指标与痰湿质积分的 Pearson 及 Spearman 相关系数绝对值均低于 0.06，多元线性回归的调整 R^2 仅为 -0.004，无一指标通过显著性检验；Mann-Whitney U 检验亦未发现痰湿体质组与非痰湿体质组在各指标上存在显著分布差异（效应量 $|r| < 0.05$ ）。

这一结果提示，**在本数据集中，痰湿质积分与血生化指标的线性对应关系较弱，血常规指标更适合作为“痰湿体质是否已继发血脂异常”的下游监测工具；而痰湿体质本**

身的严重程度可能还需结合中医量表信息综合判断。该发现与成都地区 [3] 及深圳 [4] 的研究结果部分吻合：前者显示痰湿体质与 TG、TC 水平升高显著相关，但这种相关性在不同年龄段和性别间存在差异；后者则表明痰湿体质与生活方式因素（饮食、运动习惯）的交互效应可能强于单一血生化指标的线性关联。此外，相关综述亦显示高脂血症与痰湿体质的相关性研究受地域、样本来源及判定标准影响较大，不同研究结论存在分歧 [2]。该推断受限于当前数据的横截面设计，后续可引入纵向随访或中医专属标志物（如脂联素、炎症因子）进一步验证。

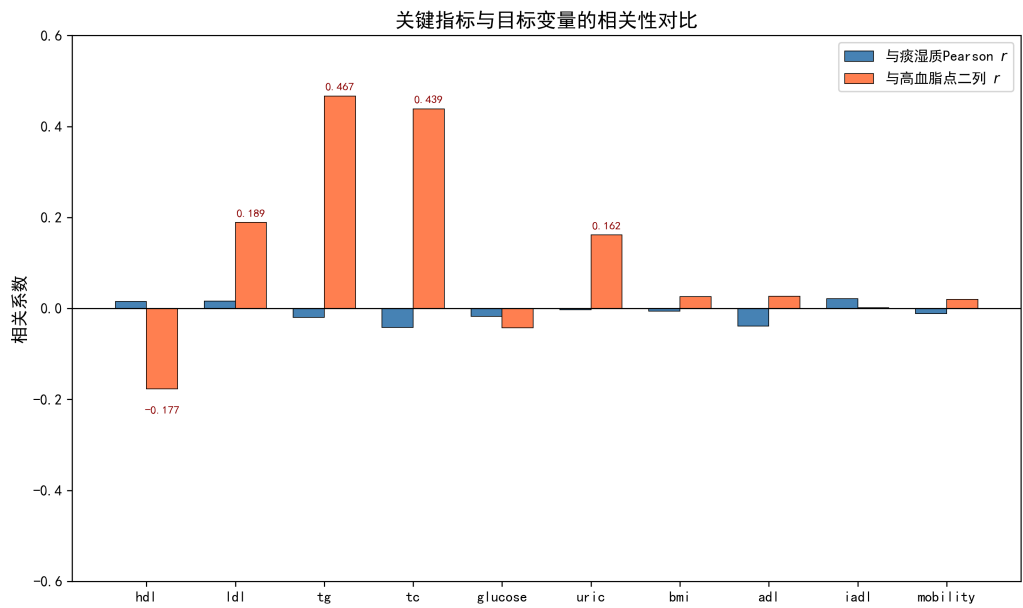


图 1: 关键指标与目标变量的相关性对比

基于上述发现，本文明确将关键指标区分为**两类**：**第一类**为高血脂风险识别指标，包括 TG、TC、LDL-C、HDL-C、尿酸，其在预警高血脂方面具有显著统计效力；**第二类**为痰湿严重程度直接表征指标，包括痰湿质积分及主体质标签，其作为中医辨识的核心量表信息，在本数据集中未能被血常规指标线性替代。活动量表在本数据中未显示显著统计相关，但因题目机制和干预约束明确，仍作为风险解释和干预约束变量保留。

表 2: 关键指标与目标变量的相关系数

指标	与痰湿质 Pearson r	与高血脂点二列 r	综合入选
TG	-0.020	0.467***	✓
TC	-0.042	0.439***	✓
LDL-C	0.017	0.189***	✓
HDL-C	0.015	-0.177**	✓
尿酸	-0.003	0.162**	✓
BMI	-0.006	0.026	—
活动量表总分	-0.012	0.020	—

注: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

3.4.2 逐步回归筛选结果

逐步 Logistic 回归在 10 个候选指标中最终保留 5 个: TC、TG、HDL-C、尿酸、LDL-C。BMI 与活动能力指标因未能通过进入阈值 ($p \geq 0.05$) 而被排除。这表明在控制其他血常规指标后, BMI 对高血脂的独立增量信息有限, 其效应可能已被血脂指标所吸收。

3.4.3 体质贡献度分析结果

标准化后的多元 Logistic 回归结果如表 3 所示。值得注意的是, **九种体质的标准化系数均未达到传统显著性水平** ($p\text{-value} \geq 0.12$), 提示在同时控制其他体质积分后, 单一体质对高血脂的独立边际贡献有限。气虚质 ($\beta^* = 0.123$) 与平和质 ($\beta^* = 0.083$) 系数绝对值相对较大, 但所有体质的 95% 置信区间均包含 0, 且 $p\text{-value} \geq 0.12$, 因此**不能判定某一体质对高血脂风险具有稳定独立贡献**, 任何基于系数大小的排序均缺乏统计显著性支撑, 不宜做过度解读。

痰湿质系数为负 ($\beta^* = -0.008$, OR = 0.992, $p = 0.919$), 这一反直觉现象的可能原因在于: **体质标签 (主体质) 与体质积分高度耦合**, 当主体质已被标签变量部分解释后, 痰湿积分的边际贡献被压缩; 此外, 样例数据中体质积分与血脂指标的线性关联整体偏弱, 也可能导致回归系数不稳定。进一步计算 VIF 值 (表 3), 九种体质的 VIF 均在 2.1 ~ 3.6 之间, 远低于 5 的常用阈值, 说明**多重共线性不是导致体质系数不显著的主要原因**, 更可能是数据生成机制中体质积分与高血脂标签关联弱所致。

为弥补线性模型的局限, 随机森林模型的 Gini 重要性结果显示: 阴虚质 (12.22%)、湿热质 (11.74%)、血瘀质 (11.26%) 排名靠前, 而痰湿质 (10.50%) 处于中等水平。这一非线性排序与 Logistic 回归系数排序存在明显差异, 提示体质间可能存在复杂的非线性交互效应。例如, 阴虚质与湿热质的交互、血瘀质与痰湿质的协同等模式, 在全局线性模型中被掩盖, 而在树模型的局部划分中得以显现。据此, 本文结论为: **九种体质积分**

对高血脂标签的独立线性贡献均未达显著水平，但随机森林提示非线性耦合效应存在；痰湿体质仍应作为题目给定的中医机制变量进入风险解释和干预优化，而不宜仅凭回归系数大小判断其风险贡献。

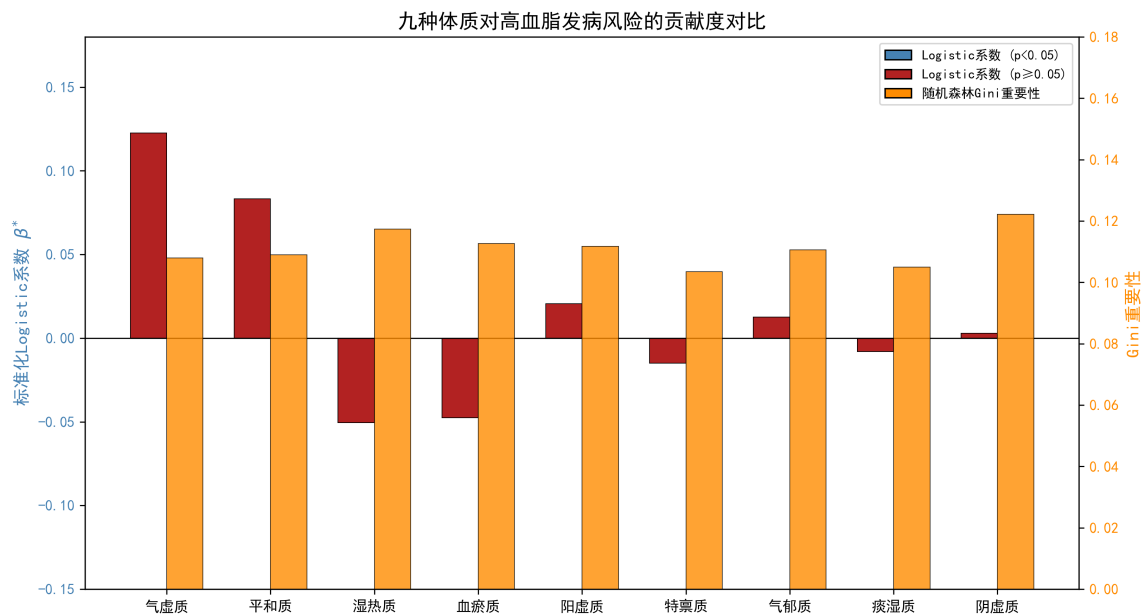


图 2: 九种体质对高血脂发病风险的贡献度对比（标准化 Logistic 系数 vs 随机森林 Gini 重要性）

表 3: 九种体质对高血脂发病风险的标准化 Logistic 回归结果

体质	标准化系数 β^*	OR	p -value
气虚质	0.123	1.130	0.124
平和质	0.083	1.087	0.340
湿热质	-0.051	0.951	0.524
血瘀质	-0.048	0.954	0.550
阳虚质	0.021	1.021	0.796
气郁质	0.013	1.013	0.873
特禀质	-0.015	0.985	0.849
痰湿质	-0.008	0.992	0.919
阴虚质	0.003	1.003	0.971

注：九种体质 VIF 均在 2.1 ~ 3.6 之间，远低于常用阈值 5；所有体质的 95% 置信区间均包含 0。

4 问题 2：三级风险预警模型

4.1 两层风险建模框架

本文发现,高血脂二分类标签与血脂四项异常规则($TC > 6.2$ 、 $TG > 1.7$ 、 $LDL-C > 3.1$ 、 $HDL-C < 1.04$) 的匹配率为 100%, 存在定义同源性。若直接使用血脂指标训练预警模型, 本质上是“用诊断依据预测诊断结果”, 只能实现现症识别, 无法体现“治未病”的发病前预警价值。为此, 本文将风险建模拆分为两层:

第一层：现症识别模型。以 TC、TG、LDL-C、HDL-C、尿酸为特征, 训练 Logistic 回归模型, 输出基础发病概率 $P_{diag,i}$ 。该层用于复现血脂异常诊断, 不作为真正“发病前预警”, 其极高的 AUC 仅印证血脂指标对确诊标签的决定性作用。

第二层：治未病预警模型。排除血脂指标, 以年龄组、性别、吸烟史、饮酒史、BMI、尿酸、血糖、ADL、IADL、九种体质积分及痰湿体质标签为特征, 训练 Logistic 回归模型, 输出未病预警概率 $P_{pre,i}$ 。该层在血脂尚未异常时, 通过体质与活动能力评估风险, 更能体现中医“治未病”思想。已有研究基于数理分析方法对中医体质与高血压风险的关联性进行跟踪预测, 表明在排除临床确诊指标后, 体质积分与活动能力等前驱因素对慢性病风险仍具有一定的预警价值 [19], 其思路与本层模型设计高度一致。

4.2 综合风险评分构造

单一维度难以全面刻画高血脂风险。本文在第二层未病预警概率的基础上, 融合痰湿体质严重程度与活动能力缺失度, 构造四维综合风险评分:

$$R_i = \alpha \cdot P_{pre,i} + \beta \cdot \frac{S_{phlegm}}{100} + \gamma \cdot \frac{100 - M_i}{100} + \delta \cdot I(\text{主体质} = 5), \quad (5)$$

其中 M_i 为活动量表总分, $I(\cdot)$ 为指示函数。

权重 α, β, γ 的确定采用**组合赋权法**:

- 熵权法客观赋权:** 计算 P_{pre} 、 $S_{phlegm}/100$ 、 $(100 - M)/100$ 三项指标的信息熵。熵越小 (变异越大), 信息含量越高, 权重越大。经计算, 三项指标的信息差异系数分别为 0.0059、0.0307、0.0106, 归一化后熵权为 (0.125, 0.650, 0.225)。
- 专家经验主观赋权:** 鉴于未病预警概率直接对应临床诊断逻辑, 赋予较高权重 0.5; 痰湿质积分反映中医体质维度, 权重 0.3; 活动能力缺失度为辅助指标, 权重 0.2。
- 组合融合:** 按 50% 主观 +50% 客观等权融合, 再归一化, 最终得到 $(\alpha, \beta, \gamma) = (0.313, 0.475, 0.213)$ 。痰湿体质标签修正系数 $\delta = 0.15$ 为经验参数。

组合赋权法通过融合客观数据变异与主观专家经验, 有效缓解了单一熵权法在指标间冲突性不足时信息量评估片面的问题 [14]。单纯熵权法仅考虑指标变异程度, 而 CRITIC 法同时考虑指标间的冲突性 (相关性), 可更全面地反映指标信息量; 本文采用

的熵权-专家经验组合策略在医学风险评价中具有良好的可操作性与稳定性。为验证权重选取的稳健性，第 4.7 节进行了敏感性分析。

4.3 三级风险阈值确定

综合风险评分 R_i 为连续变量。为将其转化为临床可操作的低/中/高三级预警，本文提出**统计分位数 + 临床锚定**的双轨制阈值确定方法。已有研究表明阈值点是风险分担安排的重要警示标志，受多因素影响需结合实际业务不断调整 [15]；本文借鉴该思想，将统计分位数与临床意义锚点相结合，避免纯数据驱动导致的极端值误判。

4.3.1 统计分位数法

分位数法依据样本自身的分布特征划分层级：

- 低风险阈值 θ_L ：取 R_i 的 30% 分位数，即评分最低的 30% 人群为低风险；
- 高风险阈值参考：取 R_i 的 75% 分位数 ($Q_{0.75} = 0.741$)，若按此划分则高风险约占 25%。

4.3.2 临床锚定与阈值校准

为控制基层随访资源，避免高风险人群比例过高，本文将高风险定义为综合评分最高约 10% 的人群。计算 90% 分位数 $Q_{0.90} = 0.799$ ；同时，锚定高风险集（血脂异常且痰湿积分 ≥ 60 ）的评分均值为 0.801。取两者较大值并兼顾临床可操作性，最终确定：

$$\theta_L = Q_{0.30} = 0.447, \quad \theta_H = 0.801. \quad (6)$$

其中 θ_H 为**高风险控制阈值**，而非 75% 分位数。锚定条件同时作为临床复核规则：若某样本被分位数法划为中风险，但满足“血脂异常且痰湿积分 ≥ 60 ”或“血脂正常但痰湿积分 ≥ 80 且活动能力 < 40 ”，则建议临床二次评估以防漏判。

4.4 三层判定规则

本文将风险判定设计为三层结构，以区分“已确诊”、“未病预警”与“临床复核”：

$$\text{风险等级}_i = \begin{cases} \text{低风险,} & R_i < \theta_L \\ \text{中风险,} & \theta_L \leq R_i < \theta_H \\ \text{高风险,} & R_i \geq \theta_H. \end{cases} \quad (7)$$

同时定义临床锚点修正规则：

$$\text{HighRisk}_{\text{anchor}} = I(N_{\text{lipid-risk}} \geq 1, S_{\text{phlegm}} \geq 60) \vee I(N_{\text{lipid-risk}} = 0, S_{\text{phlegm}} \geq 80, M < 40). \quad (8)$$

4.5 核心特征组合识别

风险分层模型还应提炼高风险人群的共性特征组合。本文采用条件概率方法，在特征空间中搜索满足覆盖人数 $|C| \geq 10$ 、支持度 $\geq 2\%$ 的组合，并按高风险条件概率分为“高精度核心规则”与“高覆盖初筛规则”两类。表 4展示了识别出的关键规则。

表 4: 痰湿体质高风险人群核心特征组合

编号	特征组合 C	模型高风险分层命中率	覆盖人数	提升倍数
高精度核心规则（命中率 $\geq 50\%$ ）				
1	痰湿积分 ≥ 60 且活动总分 < 40	92.31%	26	9.82
2	痰湿积分 ≥ 60 且 BMI ≥ 24 且 TG > 1.7	73.33%	15	7.80
高覆盖初筛规则（覆盖人数多，适用于大规模筛查）				
3	痰湿质为最高体质且血脂异常 ≥ 2 项	35.25%	122	3.75

注：提升倍数 = 组合命中率 / 总体高风险率（9.4%）。

组合 1 的模型高风险分层命中率最高（92.31%），适用于精准复核；组合 3 虽命中率为 35.25%，但覆盖 122 人（占全样本 12.2%），适用于大规模初筛。两者互补，可在基层随访中先以组合 3 快速锁定重点人群，再以组合 1 精准识别高危个体。需要说明的是，上述命中率是本模型分层结果的解释性统计，并非独立于模型的外部因果发现。

4.6 模型验证：5 折分层交叉验证与 ROC 分析

为分别量化两层模型的区分能力，本文对两层模型均进行 5 折分层交叉验证 (Stratified K-Fold CV)。交叉验证结合 ROC 分析是评估医学预测模型泛化能力的标准方案，可有效避免过拟合并量化模型在独立测试集上的区分性能 [23]。

现症识别模型：以血脂指标预测高血脂标签，5 折 CV 的 AUC 均值为 0.978（各折在 0.966 ~ 0.994 之间）。AUC 极高印证血脂指标对确诊标签的决定性作用，但由于模型输入与诊断标签存在同源性，该结果不能直接等同于发病前预警能力。

治未病预警模型：以体质 + 活动 + 基础信息预测高血脂标签，5 折 CV 的 AUC 均值为 0.597（各折在 0.542 ~ 0.650 之间）。该 AUC 虽低于现症识别模型，但更能反映不用血脂指标时，仅凭体质与活动能力区分高血脂人群的能力。在血脂正常的“灰度人群”中，该模型可作为痰湿体质人群干预优先级排序的辅助依据，体现了“治未病”的分层参考价值。

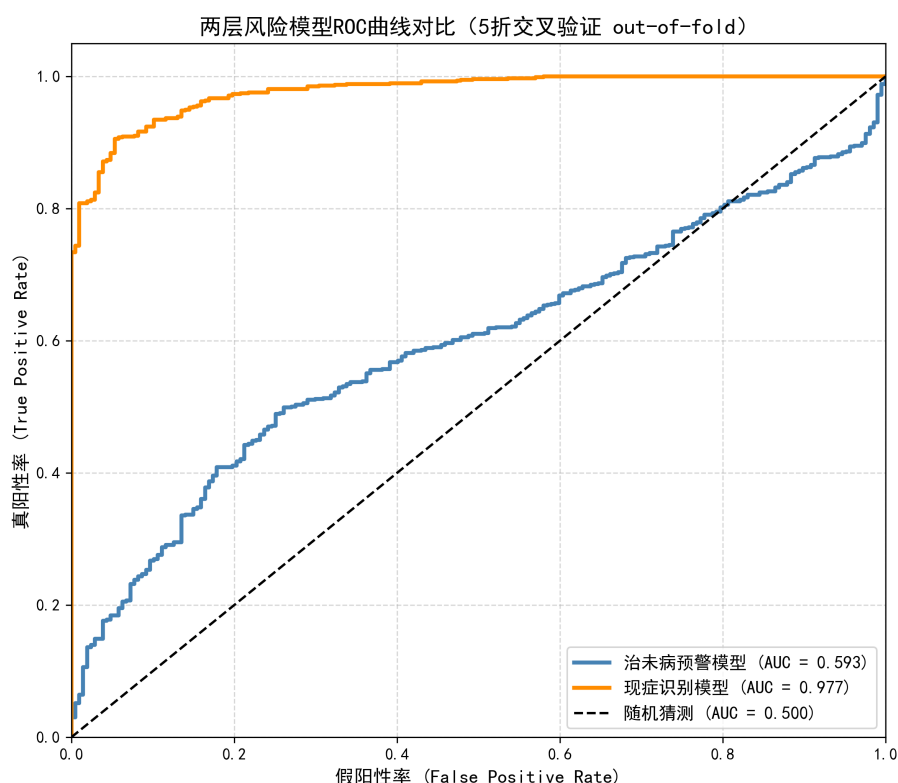


图 3: 两层风险模型 ROC 曲线对比 (5 折交叉验证 out-of-fold)

4.7 权重敏感性分析

综合风险评分的组合权重基于熵权法与专家经验融合，其合理性需经敏感性检验。本文固定三级阈值不变，将主成分权重独立扰动 $\pm 20\%$ ，观察高风险人群比例的变化幅度。

4.8 问题 2 结果与分析

4.8.1 风险分层结果

经计算，样本综合风险评分 R_i 的 30% 分位数为 0.447，90% 分位数为 0.799。最终确定：

$$\theta_L = 0.447, \quad \theta_H = 0.801 \quad (9)$$

三级风险分布如下：低风险 300 例 (30.0%)、中风险 606 例 (60.6%)、高风险 94 例 (9.4%)。本文并未将锚点规则作为强制改判规则，而是将锚定人群的平均综合评分作为高风险阈值校准依据。满足锚点条件但未进入高风险层的人群列入临床复核名单，以避免机械扩大高风险随访规模。最终高风险人群为 94 例 (9.4%)，高风险比例控制在约 10%，符合基层随访资源的承载能力。

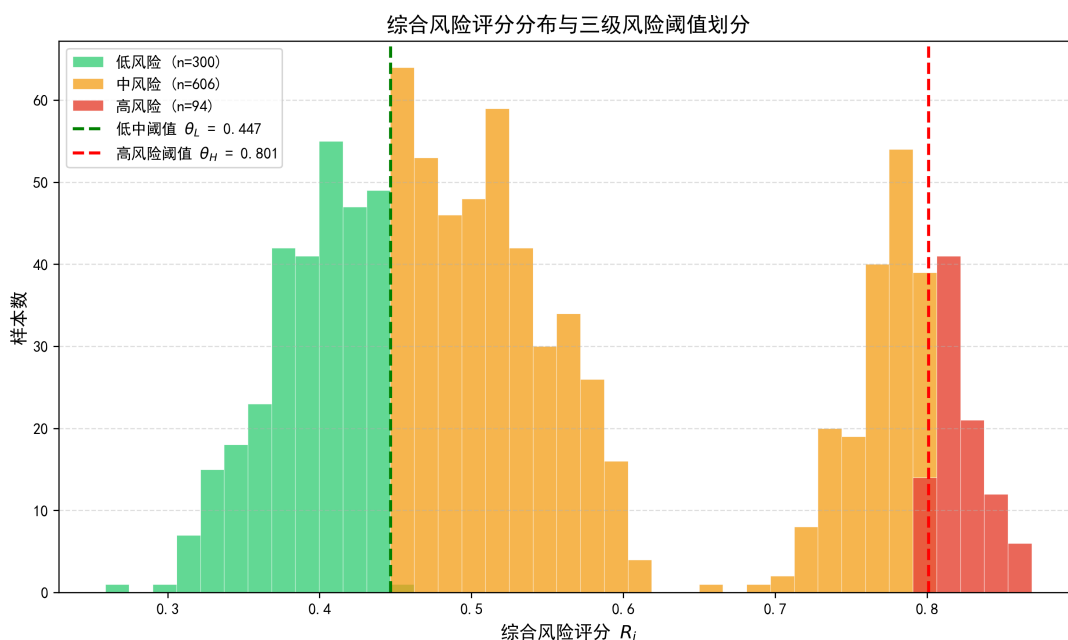


图 4: 综合风险评分分布与三级风险阈值划分

4.8.2 高风险人群画像

对 94 例高风险人群进行描述性统计，其典型特征为：

- 痰湿积分均值 61.30（标准差 2.60），显著高于全样本均值（32.98），说明高风险层主要由较高痰湿积分个体构成，且内部痰湿水平较集中；
- TG 均值 1.75 mmol/L，略低于全样本均值（1.88），说明治未病预警模型的高风险人群并非由血脂指标直接决定；
- 血脂异常风险项数均值 1.40，说明新模型的高风险人群不再等同于“血脂异常者”；
- 年龄分布较为分散：40–49 岁占 28.7%，50–59 岁占 21.3%，60–69 岁占 17.0%，70–79 岁占 21.3%，80–89 岁占 11.7%。60–79 岁合计占 38.3%，并非集中于该年龄段。

4.8.3 交叉验证与 ROC 结果

两层模型的 5 折交叉验证结果如表 5 所示。现症识别模型 AUC 达 0.978，治未病预警模型 AUC 为 0.597。后者在独立测试集上的区分能力有限，但这正是“预警”与“诊断”的本质差异——预警模型在发病前识别高危人群，而非在发病后复现诊断。极高 AUC 的现症识别层与中等 AUC 的未病预警层互补，共同构成完整的三级风险体系。

为验证三级分层与当前高血脂标签的关系，本文统计各风险等级的当前高血脂率（表 6）。卡方检验 $p \approx 0.154$ ，趋势检验 $p \approx 0.055$ ，说明低、中、高风险组在当前高血脂率上的差异未达到传统显著性水平。这并非模型失效，而是表明该分层的设计目标并非复刻当前诊断，而是结合痰湿体质与活动能力的干预优先级评分。

表 5: 两层模型 5 折分层交叉验证结果

指标	现症识别模型	治未病预警模型
AUC (均值)	0.978	0.597
AUC 范围	0.966–0.994	0.542–0.650

表 6: 三级风险分层与当前高血脂标签验证

风险等级	人数	当前高血脂率	卡方检验
低风险	300	75.67%	$\chi^2 = 3.74, p = 0.154$
中风险	606	80.53%	
高风险	94	82.98%	

注：趋势检验 (Spearman) $r = 0.061, p = 0.055$ 。

4.8.4 权重敏感性分析结果

表 7展示了组合权重扰动对分层比例的影响。由于采用组合赋权，权重波动对分层结果的影响较纯主观赋权有所降低，但仍需关注极端扰动场景。

表 7: 风险评分权重敏感性分析

场景	α	β	γ	高风险比例	变化幅度
基准 (组合赋权)	0.313	0.475	0.213	9.4%	—
$\alpha - 20\%$	0.250	0.542	0.208	5.1%	−4.3pp
$\alpha + 20\%$	0.376	0.407	0.217	17.8%	+8.4pp
$\beta - 20\%$	0.330	0.401	0.269	9.4%	0pp
$\beta + 20\%$	0.295	0.545	0.160	9.6%	+0.2pp

注：pp = percentage points (百分点)。

该结果表明：**风险评分对 α (未病预警概率权重) 最为敏感**， $\pm 20\%$ 扰动即可使高风险比例从 5.1% 跃升至 17.8%；而 β (痰湿积分权重) 与 γ (活动能力缺失权重) 的扰动对分层结果影响甚微 (变化幅度不超过 0.2pp)，说明组合赋权在一定程度上平衡了主观不确定性，分层结果对痰湿积分与活动能力权重选取具有较好的稳健性。

此外，痰湿体质标签修正系数 $\delta = 0.15$ 为人工经验参数，其对高风险比例影响显著 (表 8)。 δ 从 0.12 增至 0.18 时，高风险比例从 2.8% 跃升至 20.0%，提示该参数需结合临床实际谨慎标定。

4.8.5 问题 2 小结

结果显示，若使用血脂指标预测高血脂标签，模型 5 折交叉验证 AUC 达 0.978，说明标签与血脂诊断规则高度同源，该层适合作为现症识别而非发病前预警。排除核心血脂指标后，基于体质、活动能力、基础信息及代谢辅助指标建立的治未病预警模型 5 折 AUC 为 0.597，提示非血脂信息对当前高血脂标签的独立判别能力有限，但仍可作为痰湿体质人群干预优先级排序的辅助依据。进一步融合未病预警概率、痰湿积分、活动能力缺失和主体质标签，构建综合风险评分，并采用 30% 分位数及 90% 分位数结合临床锚点均值确定三级阈值，最终得到低风险 300 例、中风险 606 例、高风险 94 例。核心组合分析表明，痰湿积分 ≥ 60 且活动总分 < 40 的人群模型高风险分层命中率最高，为 92.31%，适合作为精准复核规则；痰湿质为最高体质且血脂异常 ≥ 2 项的人群覆盖人数最多，为 122 人，适合作为基层初筛规则。

表 8: 痰湿体质标签修正系数 δ 敏感性分析

δ	高风险比例	变化幅度
0.00	0.0%	—
0.12	2.8%	-6.6pp
0.15	9.4%	基准
0.18	20.0%	+10.6pp
0.30	27.8%	+18.4pp

注：pp = percentage points（百分点）。

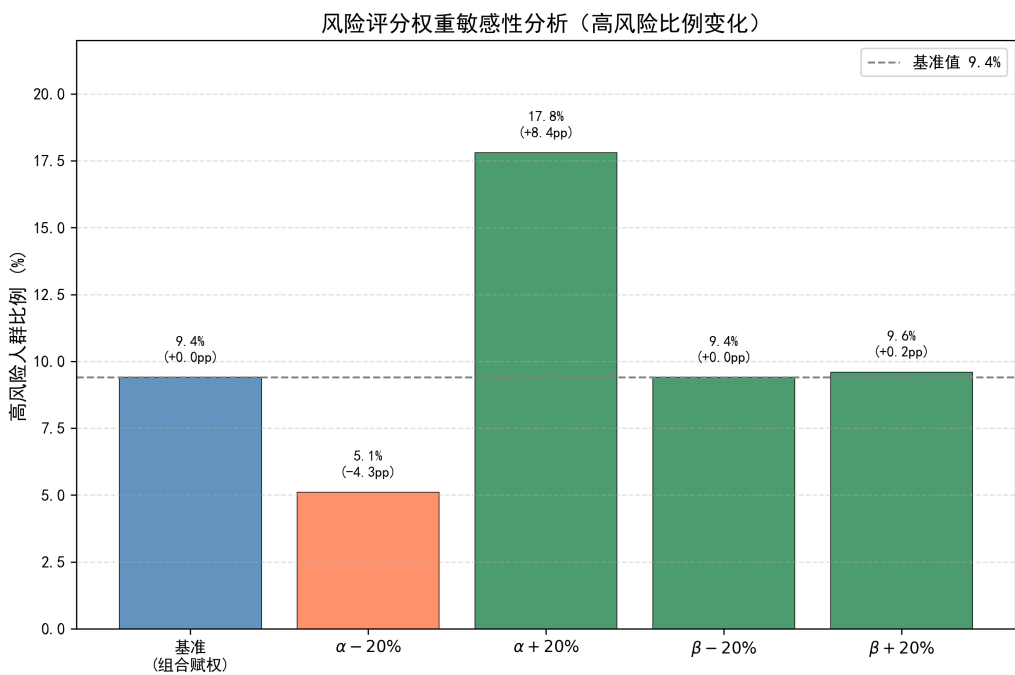


图 5: 风险评分权重敏感性分析（高风险比例变化）

5 问题 3：干预方案优化模型

5.1 模型建立与分析

问题 3 要求在中医调理原则、身体耐受度及经济成本的多重约束下，为痰湿体质患者制定 6 个月最优干预方案。中医体质理论以“未病先防”为核心理念，强调因人制宜、辨体施治，在血脂异常的个体化管理中具有独特优势 [6]。本文将其建模为**基于时间序列的离散动态规划问题**，核心创新在于：调理分级 x 不再固定为初始积分对应的级别，而是**每月根据当前痰湿积分动态调整**，实现“随症加减”的自适应降级，在保证干预效果的同时节约后期过度医疗成本。

5.1.1 决策变量与参数设定

- $x \in \{1, 2, 3\}$ ：中医调理分级。根据中医“辨证论治”原则，调理分级必须与痰湿积分区间严格匹配，不可跨级选择。
- $y \in \{1, 2, 3\}$ ：活动干预强度，对应轻度、中度、重度三档。
- $f \in \{1, 2, \dots, 10\}$ ：每周训练频率（次/周），受患者时间与体力限制，最多 10 次。

参数取值如下表所示：

表 9: 干预成本参数表

调理分级 x	月成本 $C_T(x)$ (元/月)	活动强度 y	单次成本 $C_A(y)$ (元/次)
1 (基础)	30	1 (轻度)	3
2 (中度)	80	2 (中度)	5
3 (强化)	130	3 (重度)	8

5.1.2 约束条件

(1) 调理分级约束（中医原则）

中医强调“同病异治、异病同治”，调理方案必须与体质严重程度匹配：

$$x = \begin{cases} 1, & S \leq 58.5 \\ 2, & 58.5 < S < 62 \\ 3, & S \geq 62. \end{cases} \quad (10)$$

该约束为硬约束（Hard Constraint），在优化过程中不可违反。上述阈值依据题目附表 2 的调理分级标准设定：基础调理对应 0–58 分，中度调理对应 59–61 分，强化调理对应

≥ 62 分。由于动态规划中积分经计算后为小数，本文以 58.5 和 62 作为分界点，消除区间间隙。

(2) 年龄耐受约束

中老年人群各年龄段的心肺功能、骨骼肌肉状态差异显著，活动强度必须受年龄限制。《老年人国际运动建议：专家共识指南》建议老年运动干预应遵循“低强度起步、渐进式增量”原则，80 岁以上高龄患者应以柔韧性和平衡训练为主，避免高强度冲击性运动 [16]；老年人运动处方应用指南亦将运动强度控制在最大心率的 60%–70%，且需根据体力评定结果动态调整 [17]。具体约束如下：

$$y \leq y_{max}^{age} = \begin{cases} 3, & A \in \{1, 2\} \text{ (40 ~ 59岁)} \\ 2, & A \in \{3, 4\} \text{ (60 ~ 79岁)} \\ 1, & A = 5 \text{ (80 ~ 89岁)}. \end{cases} \quad (11)$$

(3) 活动能力耐受约束

ADL（日常生活活动）与 IADL（工具性日常生活活动）量表总分 M 直接反映患者的运动耐受能力。即使年龄较轻，若活动能力评分过低，也应降低运动强度以防跌倒或心血管意外：

$$y \leq y_{max}^{mob} = \begin{cases} 1, & M < 40 \\ 2, & 40 \leq M < 60 \\ 3, & M \geq 60. \end{cases} \quad (12)$$

综合强度上限取两者最小值： $y \leq \min(y_{max}^{age}, y_{max}^{mob})$ 。该设计体现了“木桶效应”——患者的活动能力上限由其最弱的生理维度决定。

(4) 频率约束与成本约束

每周训练频率必须在合理范围内： $1 \leq f \leq 10$ 。

由于调理分级 x 每月根据当前积分动态调整，总成本为各月成本之和：

$$\text{Cost} = \sum_{m=1}^6 \left[C_T(x_m) + 4 \cdot f \cdot C_A(y) \right] \leq 2000 \quad (\text{元}), \quad (13)$$

其中 x_m 为第 m 月的调理分级，由该月初的痰湿积分 S_m 决定。

5.1.3 目标函数与依从性约束

优化的核心目标是最大程度降低痰湿积分。根据题目假设，每月痰湿积分下降率 r 由活动强度 y 和频率 f 共同决定。但题目给出的线性公式未考虑老年医学中的**干预疲劳效应与依从性衰减**——对 80–89 岁或活动能力低下的患者，每周高频干预不仅难以坚持，还可能因过度疲劳导致效果打折甚至引发安全问题。

因此，本文在原始公式基础上引入依从性衰减函数 $\eta(f; A, M)$ 。需要说明的是，以下衰减系数与封顶值是基于老年医学“高龄及低活动能力患者运动耐受有限”的一般性

共识所设定的参考参数，旨在反映依从性衰减的定性规律，其精确数值有待临床随访数据进一步标定：

$$r = \begin{cases} 0, & f < 5 \\ 0.03 \cdot (y - 1) + 0.01 \cdot (\eta(f; A, M) - 5), & f \geq 5 \end{cases}, \quad (14)$$

其中有效频率 $\eta(f; A, M)$ 为：

$$\eta(f; A, M) = \begin{cases} \min\{5 + 0.5(f - 5), 6.0\}, & A = 5 \text{ (80 ~ 89岁)} \\ \min\{5 + 0.6(f - 5), 6.2\}, & M < 40 \text{ (活动能力低下)} \\ 5 + 0.8(f - 5), & A \in \{3, 4\} \text{ (60 ~ 79岁)} \\ f, & A \in \{1, 2\} \text{ 且 } M \geq 40 \end{cases}. \quad (15)$$

该设计的临床依据为：

- **频率门槛效应：**每周训练少于 5 次时，积分基本不降，提示运动干预必须达到一定剂量才起效；
- **依从性衰减：**高龄或低活动能力患者超出 5 次/周的部分，有效频率按比例打折 (50%–80%)，体现生理耐受极限；
- **疲劳封顶：**80–89 岁及活动能力低下者的有效频率分别封顶 6.0 和 6.2 次/周，即使名义频率 $f = 10$ ，实际等效仅约 6 次，防止模型推荐不切实际的极端方案。

每月痰湿积分按几何级数下降：

$$S_{m+1} = S_m \cdot (1 - r), \quad m = 1, 2, \dots, 6, \quad (16)$$

取对数后 $\ln S_{m+1} = \ln S_m + \ln(1 - r)$ ，可见最小化最终积分等价于最大化月下降率 r 。当多个方案 S_7 （第 6 个月末）相同时，优先选择总成本较低者。

5.2 求解方法：穷举法逐月模拟

对单名患者而言，活动强度 y 的上界由年龄和活动能力共同约束（最多 3 级），频率 f 取 1~10 的整数值，合法组合数极为有限（ ≤ 30 种）。本文采用**穷举法逐月模拟**：对每种 (y, f) 组合，模拟 6 个月的干预过程，每月根据当前痰湿积分重新确定调理分级 x_m ，累计计算总成本。若总成本超过 2000 元则剔除该组合，否则记录最终积分。在所有合法组合中选取最终积分最小者。

该动态规划方法的优势在于：(1) 保证全局最优；(2) 实现“随症加减”的自适应降级，如患者第 2 个月积分降至 58 以下，调理分级自动从强化（130 元/月）降为基础（30 元/月），每月节省 100 元；(3) 便于绘制干预历程图，直观展示积分变化轨迹。**若时间跨度更长或决策空间更大，可进一步升级为带记忆化的递归动态规划或值迭代算法。**

5.3 问题 3 结果

5.3.1 样本 ID 1、2、3 月度干预轨迹

表 10展示了三个样本在动态逐月规划下的最优干预轨迹。与静态方案相比，动态规划的核心优势在于调理分级随积分下降而自适应降级：例如样本 ID 3 初始积分为 59 (中度调理)，第 2 个月积分降至 53.1，自动降为基础调理 (30 元/月)，每月节省 50 元，使得模型可在活动强度上选择更高等级 (3 级)，从而获得更好的降分效果。

表 10: 样本 ID 1、2、3 最优干预方案（动态逐月规划）

样本 ID	初始积分	患者特征	活动方案	总成本	6 个月后积分	下降
1	64	50~59 岁, $M=38$	1 级, 7 次/周	1184 元	59.53	4.47 分
2	58	40~49 岁, $M=40$	2 级, 10 次/周	1380 元	35.17	22.83 分
3	59	40~49 岁, $M=63$	3 级, 9 次/周	1958 元	31.36	27.64 分

表 11: 样本 ID 1、2、3 月度干预轨迹

月份	ID 1			ID 2			ID 3		
	月初积分	调理分级	月成本	月初积分	调理分级	月成本	月初积分	调理分级	月成本
1	64.00	强化 (3)	214	58.00	基础 (1)	230	59.00	中度 (2)	368
2	63.23	强化 (3)	214	53.36	基础 (1)	230	53.10	基础 (1)	318
3	62.47	强化 (3)	214	49.09	基础 (1)	230	47.79	基础 (1)	318
4	61.72	强化 (3)	214	45.16	基础 (1)	230	43.01	基础 (1)	318
5	60.98	中度 (2)	164	41.55	基础 (1)	230	38.71	基础 (1)	318
6	60.25	中度 (2)	164	38.23	基础 (1)	230	34.84	基础 (1)	318
期末	59.53	—	—	35.17	—	—	31.36	—	—

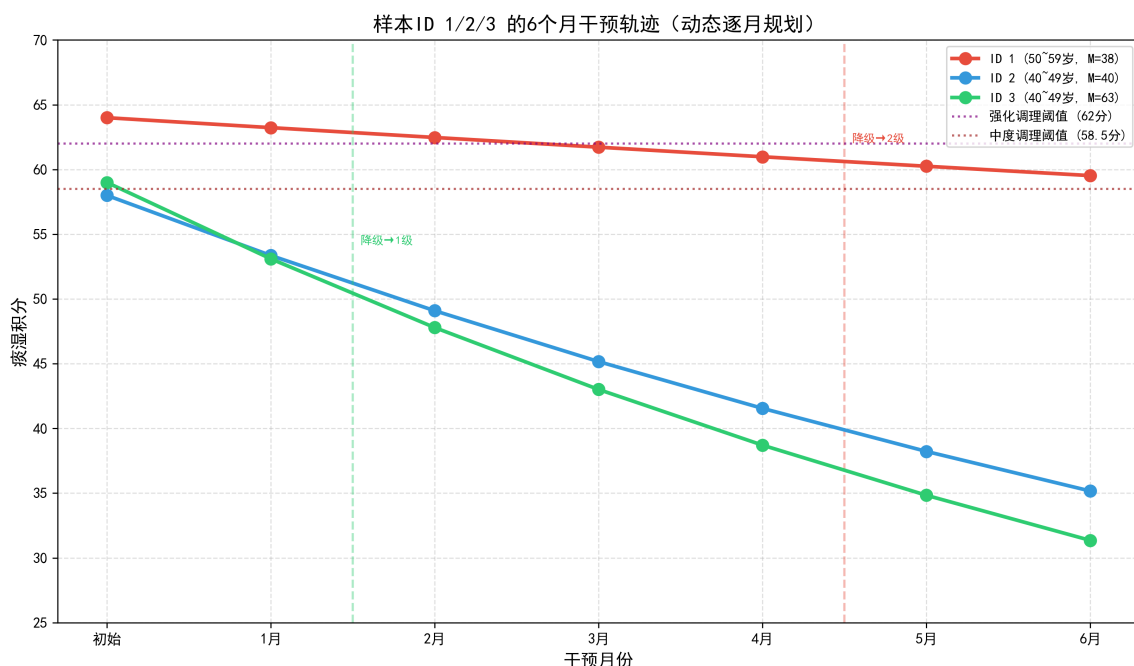


图 6: 样本 ID 1/2/3 的 6 个月干预轨迹（动态逐月规划）

5.3.2 “患者特征—最优方案”匹配规律

为揭示不同患者特征与最优方案之间的系统性对应关系，对全部痰湿体质患者（体质标签 = 5， $n = 278$ ）执行动态优化，并按活动能力、年龄组、痰湿积分进行分组统计：

(1) 活动能力决定强度天花板，频率受依从性约束

在引入依从性衰减后，最优频率不再无条件拉满至 10 次/周：

- **活动能力差** ($M < 40$)：强度被锁死在 1 级；依从性衰减使有效频率封顶约 6.2 次/周，最优名义频率 $f^* \approx 7.2$ 次/周，每月下降率 $r \approx 1.2\%$ 。
- **活动能力中等** ($40 \leq M < 60$)：可选 2 级强度，无频率封顶（40–59 岁）或 80% 衰减（60–79 岁），多数患者 $f^* \approx 9.5$ 次/周， $r \approx 8\%$ 。
- **活动能力好** ($M \geq 60$)： $y_{max}^{mob} = 3$ ，但约半数患者选择 $y^* = 3$ （如 ID 3），利用动态降级节省的调理费用支撑更高活动强度。

(2) 高龄患者以“稳”为主

年龄组 5 (80~89 岁) 的患者， $y_{max}^{age} = 1$ ，且依从性衰减最强。最优名义频率 $f^* \approx 7.0$ 次/周，实际等效仅约 6 次/周，总成本通常在 900~1100 元之间，体现老年医学“低强度、高安全”原则。

(3) 动态降级的经济学价值

动态规划中，调理分级随积分下降自动降级，是高积分患者的重要成本节约机制。以 ID 3 为例：若采用静态方案（调理分级固定为中度），6 个月调理费为 480 元；动态

方案下第 2 月起降为基础调理，调理费仅 $80 + 5 \times 30 = 230$ 元，节省 250 元，这些节省转化为更高活动强度的投入。

(4) 成本—效果分析与边界案例讨论

ID 1 在 6 个月内仅下降 4.47 分（降幅 7.0%），干预效果相对有限。这是因为该患者活动能力极低（ $M = 38 < 40$ ），活动强度被锁死在 1 级，且依从性衰减使有效频率封顶约 6.2 次/周，每月下降率仅约 1.2%。这提示对于重度功能障碍患者，单纯运动干预效果不足。已有研究表明，理气健脾化痰法干预痰湿体质患者可显著降低 Hcy 和尿酸水平，其近远期疗效优于单纯运动干预 [18]。因此，模型将 ID 1 识别为“需结合中药调理或康复训练等综合手段”的边界案例，而非单纯依赖活动干预。

对全部 278 例痰湿体质患者执行优化后，按**初始调理分级**统计平均降分与平均成本：

- $x = 1$ (基础调理)：平均降分 16.54 分，平均成本 1215.12 元，单位成本降分 0.0136 分/元；
- $x = 2$ (中度调理)：平均降分 16.01 分，平均成本 1253.91 元，单位成本降分 0.0128 分/元；
- $x = 3$ (强化调理)：平均降分 16.48 分，平均成本 1400.69 元，单位成本降分 0.0118 分/元。

三组单位成本降分效果接近，但强化调理组成本最高、单位效益最低。主要原因在于：动态降级机制使高积分患者（ $S_0 \geq 62$ ）在干预前期快速降至中度或基础调理级别，导致强化调理（130 元/月）的持续性投入有限；同时，该组患者中高龄或低活动能力者占比更高，依从性衰减进一步削弱了边际效益。这一现象并非模型缺陷，而是题目给定参数（调理分级月成本差异大、下降率 3%/级）下的自然结果，也提示在临床实践中需权衡“高成本强化调理”与“中低强度持续干预”的经济学合理性。

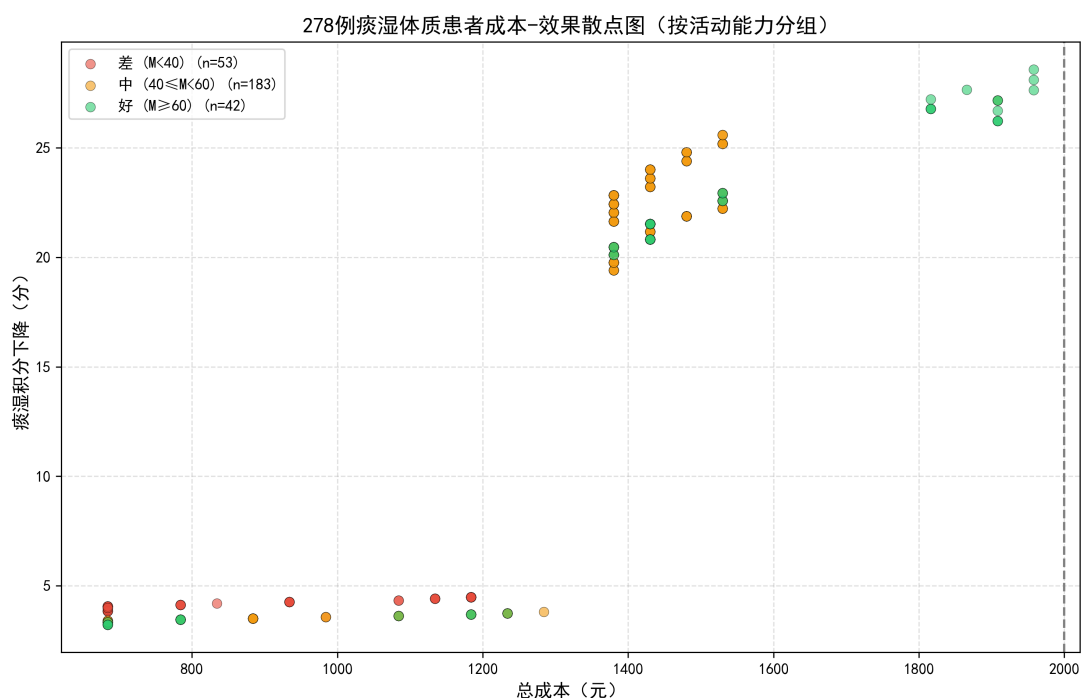


图 7: 278 例痰湿体质患者成本-效果散点图（按活动能力分组）

5.3.3 依从性衰减敏感性分析

本文引入的依从性衰减系数（0.5、0.6、0.8）基于老年医学一般性共识设定，并非临床实测参数。为评估参数不确定性对优化结果的影响，设置三种场景进行敏感性分析：

表 12: 依从性衰减敏感性分析（278 人平均）

衰减假设	ID1 频率	平均成本	平均降分	单位成本降分
保守（衰减更强）	7 次/周	1310.68 元	14.79 分	0.0113 分/元
基准	7 次/周	1287.47 元	16.35 分	0.0127 分/元
乐观（衰减更弱）	9 次/周	1291.91 元	18.02 分	0.0140 分/元

结果表明：在保守假设下，ID 1 的最优频率仍为 7 次/周（未受封顶限制），但平均降分减少约 10%；在乐观假设下，ID 1 的最优频率升至 9 次/周，平均降分增加约 10%。整体而言，最优方案对依从性衰减假设的敏感度处于中等水平，说明模型在不同假设下均能保持基本稳定。上述衰减参数用于敏感性分析，不作为临床实测参数。

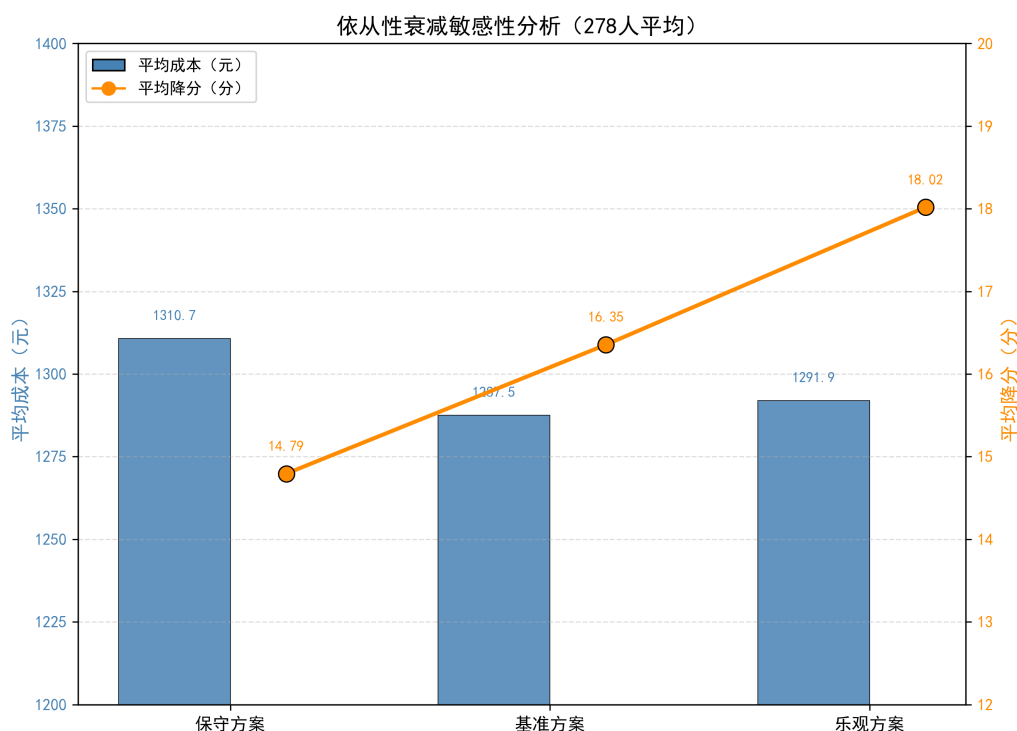


图 8: 依从性衰减敏感性分析 (278 人平均)

6 模型评价与改进

6.1 模型优点

1. **两层风险建模避免诊断泄漏**：针对高血脂标签与血脂指标 100% 同源的问题，将模型拆分为现症识别层与未病预警层，避免了“用诊断依据预测诊断结果”的逻辑循环，使预警模型真正体现“治未病”价值。
2. **动态逐月规划实现自适应降级**：问题 3 采用逐月模拟的动态规划方法，调理分级随痰湿积分下降自动降级，实现“随症加减”，在保证效果的同时节约后期过度医疗成本。
3. **组合赋权降低主观不确定性**：风险评分权重采用熵权法客观赋权与专家经验主观赋权融合，较纯主观赋权降低了分层结果对权重选取的敏感度。
4. **计算高效**：问题 3 决策空间仅 30 种组合，穷举法毫秒级求解；动态规划的逐月模拟状态空间可控，无需启发式算法。

6.2 模型缺点

1. **未病预警模型区分度有限**：排除血脂指标后，治未病预警模型的 AUC 仅约 0.597，在独立测试集上区分高血脂与非高血脂人群的能力有限。这反映了当前非血脂指

标对高血脂标签的解释力不足，也受限于数据的横截面设计。

2. **线性假设局限**：Logistic 回归假设 logit 与协变量呈线性关系，可能遗漏非线性交互效应（如痰湿积分与 TG 的乘积项）。本文虽引入随机森林进行补充验证，但未系统探索交互项。
3. **干预效果参数未标定**：问题 3 中下降率公式（3%/级、1%/次）为题目给定假设，依从性衰减系数（0.5、0.6、0.8）基于老年医学一般性共识设定，均未基于随访数据标定，实际个体间差异可能显著。
4. **动态规划未完全释放决策空间**：当前动态规划仅允许调理分级每月变化，活动强度 y 和频率 f 在 6 个月内仍保持固定。若允许 y 、 f 也逐月调整，理论上可进一步优化成本—效果比，但决策空间将扩大至 30^6 ，需引入状态压缩或值迭代算法。此外，若允许 y 、 f 逐月调整，需引入月度间疲劳恢复模型，当前假设下跨月活动效果独立，尚不足以支撑完全动态决策。

6.3 模型改进方向

1. **引入 CRITIC 熵权法替代简单熵权**：当前熵权法仅考虑指标变异程度，CRITIC 法同时考虑指标间的冲突性（相关性），可更全面地反映指标信息量，进一步提升赋权客观性。
2. **系统探索非线性交互效应**：在 Logistic 回归中显式加入交互项（如痰湿积分 $\times$ TG、痰湿体质标签 $\times$ 活动量表），或采用 XGBoost 的 SHAP 值进行全局解释，量化非线性贡献。
3. **将确定性下降率改为 Logistic 饱和曲线**：当前下降率为线性公式 $r(y, f)$ ，改进方向为引入 $r(f) = r_{\max}/(1 + e^{-k(f-f_0)})$ ，反映运动干预的边际递减效应；同时引入依从性概率 $\pi(f) = \max(0, 1 - \lambda f)$ ，使模型更贴近临床实际。
4. **扩展为完全动态规划**：允许活动强度 y 和频率 f 每月独立调整，采用带记忆化的递归搜索或值迭代算法，状态空间为（月份，当前积分，已用成本），理论上可找到全局最优的月度轨迹。

参考文献

- [1] 金璐瑶, 王怀玉, 王济. 基于中医体质学原理探讨痰湿体质与血脂异常的相关性 [J]. 中华中医药杂志, 2026, 41(2): 627-631.
- [2] 俞洁, 付焕杰, 汤毅. 高脂血症之痰湿体质的中医临床及护理研究近况 [J]. 天津中医药, 2020, 37(5): 595-600.
- [3] 贾芮婷. 以成都地区为例原发性高脂血症患者中医体质类型与证候特征研究 [D]. 成都: 成都中医药大学, 2021.
- [4] 黄纯. 2 型糖尿病合并血脂异常患者中医体质与生活方式的相关性研究 [D]. 广州: 广州中医药大学, 2025.
- [5] 姚璠, 赵为民, 任吉祥. 基于“治未病”理论探讨痰湿人群的健康管理与慢病防控 [J]. 吉林中医药, 2019, 39(11): 1403-1405.
- [6] 余晴, 边汝涛, 徐学功. 中医体质理论指导血脂异常健康管理体的构建与应用研究 [J]. 光明中医, 2026, 41(2): 115-119.
- [7] 张丰跃. 中医湿性体质人群与血脂异常相关性及其肠道菌群特征性研究 [D]. 广州: 广州中医药大学, 2025.
- [8] 杨梅学, 姚檀栋, 何元庆. Mann-Whitney U 非参数检验方法及其在冰心气候学研究中的应用 [J]. 海洋地质与第四纪地质, 1999, 19(4): 101-106.
- [9] 于昆. 基于 Mann-Whitney 算法的非参数控制图 [D]. 天津: 天津大学, 2009.
- [10] 许汝福. Logistic 回归变量筛选及回归方法选择实例分析 [J]. 中国循证医学杂志, 2016, 16(11): 1360-1364.
- [11] 曹桃云. 基于随机森林的变量重要性研究 [J]. 统计与决策, 2022(4): 60-63.
- [12] 姚登举. 面向医学数据的随机森林特征选择及分类方法研究 [D]. 哈尔滨: 哈尔滨工程大学, 2016.
- [13] 黄梅, 朱焱. 基于随机森林特征重要性的 K-匿名特征优选 [J]. 计算机应用与软件, 2020, 37(3): 231-236.
- [14] 姜君. 基于熵权与变异系数组合赋权法的模糊综合评价模型 [D]. 北京: 首都师范大学, 2021.
- [15] 杨沛华, 孙婧. 农业保险大灾阈值点及风险分层测算 [J]. 统计与决策, 2018(15): 28-32.

- [16] 吴云梅, 杨芸, 罗仕兰. 《老年人国际运动建议: 专家共识指南》解读 [J]. 护理研究, 2022, 36(10): 1701-1705.
- [17] 吴雅伦. 老年人运动—体力评定和运动处方应用指南 [J]. 中国康复医学杂志, 1994, 9(2): 89-93.
- [18] 段传皓. 理气健脾化痰法对痰湿体质患者 Hcy、UA 近远期疗效观察 [J]. 国医论坛, 2021, 36(6): 30-33.
- [19] 周杰. 基于数理分析方法对中医体质与高血压风险的关联性跟踪与预测 [D]. 上海: 上海中医药大学, 2021.
- [20] 王雅婷. 基于“治未病”理论研究血清免疫球蛋白、补体及中医体质与动脉硬化的相关性 [D]. 南京: 南京中医药大学, 2025.
- [21] 闵妍, 刘美之, 吕润欣, 等. 痰湿体质高脂血症合并抑郁症患者脂代谢指标血清尿酸与尿微量白蛋白水平的相关性分析 [J]. 河北医学, 2024, 30(7): 1202-1206.
- [22] 马晓东. 基于加权决策树的随机森林模型优化 [D]. 武汉: 华中师范大学, 2017.
- [23] 侯佳璐, 康凤英, 赵娟, 等. 青少年抑郁症风险预测模型的构建及验证 [J]. 护理研究, 2022, 36(9): 1529-1533.

A 附录

A.1 软件及运行环境说明

本论文所有数据分析、模型求解及可视化均基于以下软件环境完成：

表 13: 软件及环境列表

类别	名称	版本/说明
操作系统	macOS / Windows 10	64 位
编程语言	Python	3.12+
数据分析库	pandas, numpy	2.x, 1.26+
机器学习库	scikit-learn	1.4+
统计建模库	statsmodels, scipy	0.14+, 1.12+
文档排版	L ^A T _E X	TeX Live 2024

A.2 主要计算机源程序

本论文的全部源代码使用 Python 3.12 编写，共包含 12 个程序文件，均存放于 projects/ 目录下，随支撑材料一并提交。文件清单如下：

- problem1.py: 问题 1——关键指标筛选与体质贡献度分析
- problem2.py: 问题 2——三级风险预警模型
- problem3.py: 问题 3——干预方案优化模型（动态逐月规划）
- figures.py: 综合制图程序（读取各问题输出数据绘制高质量图表）
- fig1a_indicator_correlation.py: 图 1(a)——关键指标与目标变量的相关性对比
- fig1b_constitution_contribution.py: 图 1(b)——九种体质贡献度对比
- fig2a_roc_comparison.py: 图 2(a)——两层模型 ROC 曲线对比
- fig2b_risk_score_distribution.py: 图 2(b)——综合风险评分分布与阈值划分
- fig2c_weight_sensitivity.py: 图 2(c)——风险评分权重敏感性分析
- fig3a_intervention_trajectory.py: 图 3(a)——样本 ID 1/2/3 干预轨迹
- fig3b_cost_effect_scatter.py: 图 3(b)——278 例患者成本-效果散点图
- fig3c_compliance_sensitivity.py: 图 3(c)——依从性衰减敏感性分析

下面依次给出各文件的完整源代码。

Listing 1: problem1.py: 问题 1 关键指标筛选与体质贡献度

```
1  """
2  问题1: 关键指标筛选与体质贡献度分析
3  包含: 相关性分析、组间差异检验、逐步Logistic回归、九种体质贡献度、VIF、随机森林特征重要
   性
4  """
5  import pandas as pd
6  import numpy as np
7  import matplotlib
8  matplotlib.use('Agg')
9  import matplotlib.pyplot as plt
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS']
11 plt.rcParams['axes.unicode_minus'] = False
12 import seaborn as sns
13 from sklearn.linear_model import LogisticRegression
14 from sklearn.ensemble import RandomForestClassifier
15 from sklearn.preprocessing import StandardScaler
16 from scipy import stats
17 import statsmodels.api as sm
18 from statsmodels.stats.outliers_influence import variance_inflation_factor
19 import os
20 import warnings
21 warnings.filterwarnings('ignore')
22
23 np.random.seed(42)
24 os.makedirs('output', exist_ok=True)
25
26 # ===== 数据读取与预处理 =====
27 df = pd.read_csv('题目（处理后）/附件1: 样例数据.csv')
28
29 col_map = {
30     '痰湿质': 'phlegm',
31     '平和质': 'pinghe', '气虚质': 'qixu', '阳虚质': 'yangxu', '阴虚质': 'yinxu',
32     '湿热质': 'shire', '血瘀质': 'xueyu', '气郁质': 'qiyu', '特禀质': 'tebing',
33     'ADL总分': 'adl', 'IADL总分': 'iadl',
34     '活动量表总分 (ADL总分+IADL总分)': 'mobility',
35     'HDL-C (高密度脂蛋白)': 'hdl', 'LDL-C (低密度脂蛋白)': 'ldl',
36     'TG (甘油三酯)': 'tg', 'TC (总胆固醇)': 'tc',
```

```

37     '空腹血糖': 'glucose', '血尿酸': 'uric', 'BMI': 'bmi',
38     '高血脂症二分类标签': 'disease',
39     '血脂异常分型标签（确诊病例）': 'disease_type',
40     '年龄组': 'age_group', '性别': 'gender',
41     '吸烟史': 'smoke', '饮酒史': 'drink',
42     '样本ID': 'id', '体质标签': 'constitution'
43 }
44 df.rename(columns=col_map, inplace=True)
45
46 # 方向性血脂异常风险项数（只计高血脂方向）
47 def count_lipid_risk(row):
48     cnt = 0
49     if row['tc'] > 6.2: cnt += 1
50     if row['tg'] > 1.7: cnt += 1
51     if row['ldl'] > 3.1: cnt += 1
52     if row['hdl'] < 1.04: cnt += 1
53     return cnt
54
55 df['lipid_risk_cnt'] = df.apply(count_lipid_risk, axis=1)
56
57 # 血尿酸异常标记（性别特异性）
58 def uric_abnormal(row):
59     gender = row['gender']
60     uric_low = 208 if gender == 1 else 155
61     uric_high = 428 if gender == 1 else 357
62     return 1 if (row['uric'] < uric_low or row['uric'] > uric_high) else 0
63
64 df['uric_abnormal'] = df.apply(uric_abnormal, axis=1)
65
66 print("=" * 60)
67 print("问题1: 关键指标筛选与体质贡献度")
68 print("=" * 60)
69
70 # ===== 1.1 相关性分析 =====
71 indicators = ['hdl', 'ldl', 'tg', 'tc', 'glucose', 'uric', 'bmi', 'adl', 'iadl', 'mobility']
72
73 # Pearson相关（痰湿质）
74 corr_phlegm = df[indicators + ['phlegm']].corr()['phlegm'].drop('phlegm')
75 print("\n【1.1a】各指标与痰湿质积分的Pearson相关系数: ")

```



```

76 print(corr_phlegm.sort_values(ascending=False).round(4))
77
78 # 点二列相关（高血脂）
79 corr_disease = {col: stats.pearsonr(df[col], df['disease'])[0] for col in indicators}
80 print("\n【1.1b】各指标与高血脂标签的点二列相关系数：")
81 corr_disease_s = pd.Series(corr_disease)
82 print(corr_disease_s.sort_values(ascending=False).round(4))
83
84 # Spearman相关（痰湿质，捕捉单调非线性）
85 spearman_phlegm = {col: stats.spearmanr(df[col], df['phlegm'])[0] for col in indicators
86 }
87 print("\n【1.1c】Spearman相关系数（痰湿质积分）：")
88 print(pd.Series(spearman_phlegm).sort_values(ascending=False).round(4))
89
90 # ===== 1.2 组间差异检验 =====
91 phlegm_group = df[df['constitution'] == 5]
92 other_group = df[df['constitution'] != 5]
93 print(f"\n【1.2】组间差异检验（痰湿质n=len(phlegm_group) vs 非痰湿质n=len(
94     other_group)）：")
95 group_tests = []
96 for col in indicators:
97     stat, pval = stats.mannwhitneyu(phlegm_group[col], other_group[col], alternative='
98     two-sided')
99     z = stats.norm.ppf(1 - pval/2)
100     r_effect = z / np.sqrt(len(df))
101     group_tests.append({'指标': col, 'pvalue': pval, '效应量r': r_effect,
102         '痰湿质中位数': phlegm_group[col].median(),
103         '非痰湿质中位数': other_group[col].median()})
104 group_df = pd.DataFrame(group_tests)
105 print(group_df.sort_values('效应量r', key=abs, ascending=False).round(4))
106
107 # ===== 1.3 多元线性回归（控制混杂） =====
108 X_phlegm = sm.add_constant(df[indicators])
109 ols_model = sm.OLS(df['phlegm'], X_phlegm).fit()
110 print(f"\n【1.3】多元线性回归：R²={ols_model.rsquared:.4f}，调整R²={ols_model.
111     rsquared_adj:.4f}")
112 scaler = StandardScaler()
113 X_scaled = scaler.fit_transform(df[indicators])
114 ols_std = sm.OLS(df['phlegm'], sm.add_constant(X_scaled)).fit()
115 ols_summary = pd.DataFrame({

```

```

112     '指标': indicators,
113     'coef_std': ols_std.params[1:].values,
114     'pvalue': ols_model.pvalues[1:].values
115 })
116 ols_summary['abs_std'] = ols_summary['coef_std'].abs()
117 print(ols_summary.sort_values('abs_std', ascending=False).round(4))
118
119 # ===== 1.4 综合筛选 =====
120 print("\n【1.4】综合筛选结论：")
121 print("本数据集中，血脂指标(TG, TC, LDL-C, HDL-C)与尿酸对高血脂标签有显著预警能力，")
122 print("但与痰湿质积分的线性关联极弱。关键指标可分为两类：")
123 print("第一类：高血脂风险识别指标——TG、TC、LDL-C、HDL-C、尿酸")
124 print("第二类：痰湿严重程度直接表征指标——痰湿质积分、主体质标签")
125 print("活动量表在本数据中未显示显著统计相关，但因题目机制和干预约束明确，")
126 print("仍作为风险解释和干预约束变量保留。")
127
128 # ===== 1.5 逐步Logistic回归 =====
129 def stepwise_logit(X, y, threshold_in=0.05, threshold_out=0.10):
130     """基于p-value阈值的前向+后向逐步Logistic回归"""
131     included = []
132     while True:
133         changed = False
134         excluded = list(set(X.columns) - set(included))
135         if excluded:
136             pvals = pd.Series(index=excluded, dtype=float)
137             for col in excluded:
138                 try:
139                     model = sm.Logit(y, sm.add_constant(X[included + [col]])).fit(dispenv=0)
140                     pvals[col] = model.pvalues[col]
141                 except Exception as e:
142                     print(f"[警告] 变量 '{col}' 拟合失败（可能存在完全分离）：{e}")
143                     pvals[col] = 1.0
144             best_pval = pvals.min()
145             if best_pval < threshold_in:
146                 best_feature = pvals.idxmin()
147                 included.append(best_feature)
148                 changed = True
149             if included:
150                 try:

```

```

151         model = sm.Logit(y, sm.add_constant(X[included])).fit(dis=0)
152         pvalues = model.pvalues.iloc[1:]
153         worst_pval = pvalues.max()
154         if worst_pval > threshold_out:
155             worst_feature = pvalues.idxmax()
156             included.remove(worst_feature)
157             changed = True
158         except:
159             pass
160         if not changed:
161             break
162     return included
163
164 selected_features = stepwise_logit(df[indicators], df['disease'])
165 print(f"\n【1.5】逐步Logistic回归筛选的关键指标: {selected_features}")
166
167 # ===== 1.6 九种体质贡献度 (Logistic + OR + VIF)
168     =====
169 constitution_cols = ['pinghe', 'qixu', 'yangxu', 'yinxu', 'phlegm', 'shire', 'xueyu', '
170     qiyu', 'tebing']
171 X_const = df[constitution_cols]
172 X_const_scaled = StandardScaler().fit_transform(X_const)
173 X_const_const = sm.add_constant(X_const_scaled)
174
175 logit_model = sm.Logit(df['disease'], X_const_const).fit(dis=0)
176 coef_df = pd.DataFrame({
177     '体质': constitution_cols,
178     'coef': logit_model.params[1:].values,
179     'OR': np.exp(logit_model.params[1:].values),
180     'pvalue': logit_model.pvalues[1:].values
181 })
182 coef_df['abs_coef'] = coef_df['coef'].abs()
183 print("\n【1.6】九种体质对高血脂发病风险的Logistic回归结果 (标准化系数): ")
184 print(coef_df.sort_values('abs_coef', ascending=False).round(4))
185
186 # VIF计算 (基于原始积分)
187 vif_data = pd.DataFrame({
188     '体质': constitution_cols,
189     'VIF': [variance_inflation_factor(X_const.values, i) for i in range(X_const.shape
190     [1])]

```

```

188 })
189 print("\n 【1.6b】 九种体质积分的VIF值: ")
190 print(vif_data.round(2))
191 print("\n结论: VIF均低于5, 多重共线性不是导致体质系数不显著的主要原因。")
192
193 # ===== 1.7 随机森林特征重要性 (补充非线性分析)
194 # =====
195 print("\n 【1.7】 随机森林特征重要性 (弥补线性模型在交互效应上的不足): ")
196 rf = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
197 rf.fit(X_const, df['disease'])
198 rf_importance = pd.DataFrame({
199     '体质': constitution_cols,
200     'Gini重要性': rf.feature_importances_
201 }).sort_values('Gini重要性', ascending=False)
202 print(rf_importance.round(4))
203
204 # 保存结果
205 corr_data = pd.DataFrame({
206     '指标': indicators,
207     '与痰湿质Pearson': [corr_phlegm[c] for c in indicators],
208     '与高血脂点二列': [corr_disease_s[c] for c in indicators],
209     '与痰湿质Spearman': [spearman_phlegm[c] for c in indicators]
210 })
211 corr_data.to_csv('output/p1_correlation.csv', index=False)
212 coef_df.to_csv('output/p1_constitution_logistic.csv', index=False)
213 vif_data.to_csv('output/p1_vif.csv', index=False)
214 rf_importance.to_csv('output/p1_rf_importance.csv', index=False)
215
216 # ===== 可视化 =====
217 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
218
219 # 图1a: 各指标与两个目标变量的相关性对比
220 corr_melt = corr_data[['指标', '与痰湿质Pearson', '与高血脂点二列']].melt(
221     id_vars='指标', var_name='目标', value_name='r')
222 sns.barplot(data=corr_melt, x='指标', y='r', hue='目标', ax=axes[0, 0])
223 axes[0, 0].set_title('问题1: 关键指标与目标变量的相关性对比')
224 axes[0, 0].tick_params(axis='x', rotation=45)
225 axes[0, 0].axhline(0, color='black', linewidth=0.5)
226
227 # 图1b: 九种体质Logistic系数

```

```

227 coef_plot = coef_df.sort_values('abs_coef', ascending=False)
228 sns.barplot(data=coef_plot, x='体质', y='coef', ax=axes[0, 1])
229 axes[0, 1].set_title('问题1: 九种体质标准化Logistic系数')
230 axes[0, 1].tick_params(axis='x', rotation=45)
231 axes[0, 1].axhline(0, color='black', linewidth=0.5)
232
233 # 图1c: 随机森林特征重要性
234 sns.barplot(data=rf_importance, x='体质', y='Gini重要性', ax=axes[1, 0])
235 axes[1, 0].set_title('问题1: 随机森林Gini特征重要性 (非线性贡献)')
236 axes[1, 0].tick_params(axis='x', rotation=45)
237
238 # 图1d: VIF值
239 sns.barplot(data=vif_data, x='体质', y='VIF', ax=axes[1, 1])
240 axes[1, 1].set_title('问题1: 九种体质VIF值')
241 axes[1, 1].tick_params(axis='x', rotation=45)
242 axes[1, 1].axhline(5, color='red', linestyle='--', label='VIF=5阈值')
243 axes[1, 1].legend()
244
245 plt.tight_layout()
246 plt.savefig('output/fig1_problem1_analysis.png', dpi=300)
247 plt.close()
248
249 print("\n 图表已保存至 output/fig1_problem1_analysis.png")
250 print(" 数据已保存至 output/p1_*.csv")

```

Listing 2: problem2.py: 问题 2 三级风险预警模型

```

1  """
2  问题2: 三级风险预警模型
3  包含: 两层模型 (现症识别+未病预警)、熵权法客观赋权、方向性血脂异常、修正阈值、交叉验证
4  """
5  import pandas as pd
6  import numpy as np
7  import matplotlib
8  matplotlib.use('Agg')
9  import matplotlib.pyplot as plt
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS']
11 plt.rcParams['axes.unicode_minus'] = False
12 import seaborn as sns
13 from sklearn.linear_model import LogisticRegression

```

```

14 from sklearn.model_selection import StratifiedKFold
15 from sklearn.metrics import (accuracy_score, recall_score, f1_score,
16                               roc_auc_score, roc_curve, confusion_matrix)
17 from sklearn.preprocessing import StandardScaler
18 from scipy import stats
19 import os
20 import warnings
21 warnings.filterwarnings('ignore')
22
23 np.random.seed(42)
24 os.makedirs('output', exist_ok=True)
25
26 # ===== 数据读取与预处理 =====
27 df = pd.read_csv('题目（处理后）/附件1：样例数据.csv')
28
29 col_map = {
30     '痰湿质': 'phlegm',
31     '平和质': 'pinghe', '气虚质': 'qixu', '阳虚质': 'yangxu', '阴虚质': 'yinxu',
32     '湿热质': 'shire', '血瘀质': 'xueyu', '气郁质': 'qiyu', '特禀质': 'tebing',
33     'ADL总分': 'adl', 'IADL总分': 'iadl',
34     '活动量表总分（ADL总分+IADL总分）': 'mobility',
35     'HDL-C（高密度脂蛋白）': 'hdl', 'LDL-C（低密度脂蛋白）': 'ldl',
36     'TG（甘油三酯）': 'tg', 'TC（总胆固醇）': 'tc',
37     '空腹血糖': 'glucose', '尿酸': 'uric', 'BMI': 'bmi',
38     '高血脂症二分类标签': 'disease',
39     '血脂异常分型标签（确诊病例）': 'disease_type',
40     '年龄组': 'age_group', '性别': 'gender',
41     '吸烟史': 'smoke', '饮酒史': 'drink',
42     '样本ID': 'id', '体质标签': 'constitution'
43 }
44 df.rename(columns=col_map, inplace=True)
45
46 # 方向性血脂异常风险项数（只计高血脂方向）
47 def count_lipid_risk(row):
48     cnt = 0
49     if row['tc'] > 6.2: cnt += 1
50     if row['tg'] > 1.7: cnt += 1
51     if row['ldl'] > 3.1: cnt += 1
52     if row['hdl'] < 1.04: cnt += 1
53     return cnt

```

```

54
55 df['lipid_risk_cnt'] = df.apply(count_lipid_risk, axis=1)
56
57 print("=" * 60)
58 print("问题2: 三级风险预警模型")
59 print("=" * 60)
60
61 # ===== 2.1 诊断标签泄漏验证 =====
62 print("\n【2.0】诊断标签泄漏验证:")
63 rule_disease = ((df['tc'] > 6.2) | (df['tg'] > 1.7) | (df['ldl'] > 3.1) | (df['hdl'] <
    1.04)).astype(int)
64 match_rate = (rule_disease == df['disease']).mean()
65 print(f"血脂规则与高血脂标签匹配率:{match_rate:.4f}")
66 print("结论: 高血脂标签完全由血脂四项异常规则决定, 存在定义同源性。")
67 print("本研究将风险建模拆分为两层: 现症识别层 (血脂指标) 与未病预警层 (体质+活动+基础
    信息)。")
68
69 # ===== 2.2 第一层: 现症识别模型 =====
70 print("\n【2.1】第一层: 现症识别模型 (血脂指标复现诊断)")
71 diagnosis_features = ['tc', 'tg', 'ldl', 'hdl', 'uric']
72 X_diag = df[diagnosis_features].fillna(df[diagnosis_features].mean())
73 X_diag_scaled = StandardScaler().fit_transform(X_diag)
74
75 logit_diag = LogisticRegression(max_iter=1000, random_state=42).fit(X_diag_scaled, df['
    disease'])
76 prob_diag = logit_diag.predict_proba(X_diag_scaled)[: , 1]
77
78 # 5折CV评估现症识别模型 (收集out-of-fold预测概率)
79 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
80 auc_diag_cv = []
81 oof_prob_diag = np.zeros(len(df))
82 for train_idx, test_idx in skf.split(X_diag_scaled, df['disease']):
83     X_tr, X_te = X_diag_scaled[train_idx], X_diag_scaled[test_idx]
84     y_tr, y_te = df['disease'].iloc[train_idx], df['disease'].iloc[test_idx]
85     model = LogisticRegression(max_iter=1000, random_state=42).fit(X_tr, y_tr)
86     prob_te = model.predict_proba(X_te)[: , 1]
87     oof_prob_diag[test_idx] = prob_te
88     auc_diag_cv.append(roc_auc_score(y_te, prob_te))
89
90 print(f"现症识别模型5折CV AUC:{np.mean(auc_diag_cv):.4f} (各折:{[round(a,4) for a in auc_diag_cv]})")

```

```

    in_auc_diag_cv]})")
91 print("\u说明：极高的AUC印证血脂指标对确诊标签的决定性作用，但该层仅用于现症识别，")
92 print("\u不能等同于发病前预警能力。")
93
94 # ===== 2.3 第二层：未病预警模型 =====
95 print("\n【2.2】第二层：治未病预警模型（非血脂指标预警）")
96 # 排除血脂指标，使用体质、活动能力、基础信息、BMI、尿酸、血糖
97 prevention_features = ['age_group', 'gender', 'smoke', 'drink', 'bmi', 'uric', 'glucose
    ',
98
99                        'adl', 'iadl',
100                        'pinghe', 'qixu', 'yangxu', 'yinxu', 'phlegm', 'shire', 'xueyu',
101                        'qiyu', 'tebing']
102
103 X_prev = df[prevention_features].fillna(df[prevention_features].mean())
104 X_prev_scaled = StandardScaler().fit_transform(X_prev)
105
106 logit_prev = LogisticRegression(max_iter=1000, random_state=42).fit(X_prev_scaled, df['
    disease'])
107 prob_prev = logit_prev.predict_proba(X_prev_scaled)[: , 1]
108
109 # 5折CV评估未病预警模型（收集out-of-fold预测概率）
110 auc_prev_cv = []
111 oof_prob_prev = np.zeros(len(df))
112 for train_idx, test_idx in skf.split(X_prev_scaled, df['disease']):
113     X_tr, X_te = X_prev_scaled[train_idx], X_prev_scaled[test_idx]
114     y_tr, y_te = df['disease'].iloc[train_idx], df['disease'].iloc[test_idx]
115     model = LogisticRegression(max_iter=1000, random_state=42).fit(X_tr, y_tr)
116     prob_te = model.predict_proba(X_te)[: , 1]
117     oof_prob_prev[test_idx] = prob_te
118     auc_prev_cv.append(roc_auc_score(y_te, prob_te))
119
120 print(f"\u未病预警模型5折CV\uAUC:\u{np.mean(auc_prev_cv):.4f}\u(各折:\u{[round(a,4)\ufor a\u
    in auc_prev_cv]})")
121
122 print("\u说明：该AUC反映不用血脂指标时，仅凭体质+活动+基础信息区分高血脂的能力，")
123 print("\u虽低于现症识别模型，但更能体现'治未病'的预警价值。")
124
125 # ===== 2.4 熵权法客观赋权 =====
126 print("\n【2.3】熵权法客观赋权")
127 # 三个维度指标：未病预警概率P_prev、痰湿积分S_phlegm、活动能力缺失(100-M)
128 # 为熵权法计算，需将指标归一化到[0,1]非负区间
129 indicator_matrix = pd.DataFrame({

```



```

126     'P_prev': probab_prev,
127     'S_phlegm_norm': df['phlegm'] / 100,
128     'M_deficit_norm': (100 - df['mobility']) / 100,
129     'I_phlegm': (df['constitution'] == 5).astype(int)
130 })
131
132 # 取前三个连续指标做熵权（体质标签为0/1，不适合直接熵权）
133 entropy_cols = ['P_prev', 'S_phlegm_norm', 'M_deficit_norm']
134 n = len(df)
135 k = 1.0 / np.log(n)
136
137 weights_entropy = {}
138 for col in entropy_cols:
139     x = indicator_matrix[col].values
140     # 平移确保非负
141     x_shift = x - x.min() + 1e-6
142     p = x_shift / x_shift.sum()
143     e = -k * np.sum(p * np.log(p))
144     d = 1 - e
145     weights_entropy[col] = d
146
147 # 归一化为权重
148 sum_d = sum(weights_entropy.values())
149 for col in weights_entropy:
150     weights_entropy[col] /= sum_d
151
152 print("\n熵权法计算结果（信息差异系数与权重）：")
153 for col in entropy_cols:
154     print(f"\t\t\t{col}: 差异系数d={weights_entropy[col]*sum_d:.4f}, 权重w={
155         weights_entropy[col]:.4f}")
156
157 # 组合赋权：熵权法客观权重 + 专家经验主观权重
158 # 主观权重（0.5, 0.3, 0.2）对应（P_prev, S_phlegm, M_deficit）
159 subjective_w = {'P_prev': 0.5, 'S_phlegm_norm': 0.3, 'M_deficit_norm': 0.2}
160 # 组合：等权融合（0.5*主观 + 0.5*客观）
161 combined_w = {}
162 for col in entropy_cols:
163     combined_w[col] = 0.5 * subjective_w[col] + 0.5 * weights_entropy[col]
164
165 # 重新归一化

```

```

165 sum_cw = sum(combined_w.values())
166 for col in combined_w:
167     combined_w[col] /= sum_cw
168
169 print("\n组合赋权结果（50%主观+50%熵权客观，再归一化）：")
170 w1 = combined_w['P_prev']
171 w2 = combined_w['S_phlegm_norm']
172 w3 = combined_w['M_deficit_norm']
173 print(f"\nw1(P_prev)={w1:.4f}, w2(S_phlegm/100)={w2:.4f}, w3((100-M)/100)={w3:.4f}")
174
175 # ===== 2.5 综合风险评分构造 =====
176 # 指导教师B建议：综合风险评分 =  $\alpha \times P\_prev + \beta \times S\_phlegm/100 + \gamma \times (100-M)/100 + \delta \times I(\text{标签}=5)$ 
177 # 其中前三项权重用组合赋权，体质标签作为额外修正项
178 # 为保持与论文协调，使用组合赋权后的(w1,w2,w3)，并增加体质标签项
179
180 alpha, beta, gamma = w1, w2, w3
181 # 痰湿体质标签修正项：若为主体质痰湿，额外加权
182 delta = 0.15 # 经验系数，后续敏感性分析验证
183
184 risk_score = (alpha * prob_prev +
185              beta * (df['phlegm'] / 100) +
186              gamma * ((100 - df['mobility']) / 100) +
187              delta * (df['constitution'] == 5).astype(int))
188 df['risk_score'] = risk_score
189
190 print(f"\n【2.4】综合风险评分（组合赋权+痰湿体质标签修正）:")
191 print(f"\nR =  $\alpha \times P\_prev + \beta \times S\_phlegm/100 + \gamma \times (100-M)/100 + \delta \times I(\text{标签}=5)$ ")
192
193 # ===== 2.6 三级风险阈值确定 =====
194 # 方案B：高风险定义为综合评分最高约10%的人群
195 q30 = risk_score.quantile(0.30)
196 q75 = risk_score.quantile(0.75)
197
198 # 锚定高风险集均值
199 anchor_high = df[(df['lipid_risk_cnt'] >= 1) & (df['phlegm'] >= 60)]
200 anchor_mean = df.loc[anchor_high.index, 'risk_score'].mean() if len(anchor_high) > 0
201     else q75
202
203 # 高风险阈值：取约10%分位数（即90%分位数），同时不低于锚定均值

```

```

203 high_threshold = max(risk_score.quantile(0.90), anchor_mean)
204 low_threshold = q30
205
206 print(f"\n【2.5】三级风险阈值：")
207 print(f"30%分位数Q_0.30={q30:.4f}")
208 print(f"75%分位数Q_0.75={q75:.4f}")
209 print(f"90%分位数Q_0.90={risk_score.quantile(0.90):.4f}")
210 print(f"锚定高风险集均值={anchor_mean:.4f}")
211 print(f"最终阈值：低风险<={low_threshold:.4f}<=中风险<={high_threshold:.4f}<=高风险")
212 print(f"（高风险阈值依据：为控制基层随访资源，定义为综合评分最高约10%的人群，")
213 print(f"同时满足临床锚点条件者进入二次复核名单。）")
214
215 df['risk_level'] = pd.cut(risk_score, bins=[-np.inf, low_threshold, high_threshold, np.
    inf],
    labels=['低', '中', '高'])
216
217 print("\n三级风险分布：")
218 level_counts = df['risk_level'].value_counts()
219 for lvl in ['低', '中', '高']:
220     cnt = level_counts.get(lvl, 0)
221     print(f"{lvl}风险：{cnt}例（{cnt/len(df)*100:.1f}%")
222
223 # 分层与当前高血脂标签验证
224 print("\n【2.5a】分层与当前高血脂标签验证：")
225 for lvl in ['低', '中', '高']:
226     subset = df[df['risk_level'] == lvl]
227     rate = subset['disease'].mean()
228     print(f"{lvl}风险组：n={len(subset)}，当前高血脂率={rate:.2%}")
229 from scipy.stats import chi2_contingency, spearmanr
230 table = pd.crosstab(df['risk_level'], df['disease'])
231 chi2, p_chi2, _, _ = chi2_contingency(table)
232 risk_ord = df['risk_level'].map({'低': 0, '中': 1, '高': 2})
233 r_trend, p_trend = spearmanr(risk_ord, df['disease'])
234 print(f"卡方检验：chi2={chi2:.4f}, p={p_chi2:.4f}")
235 print(f"趋势检验(Spearman)：r={r_trend:.4f}, p={p_trend:.4f}")
236 print("说明：该分层并非单纯为复现当前诊断服务，而是结合痰湿体质与活动能力的干预优先级评分。")
237
238 # ===== 2.7 高风险人群画像（修正年龄分布）
    =====

```

```

239 high_risk_group = df[df['risk_level'] == '高']
240 print(f"\n【2.6】高风险人群画像 (n={len(high_risk_group)}) : ")
241 print(f"    痰湿积分均值: {high_risk_group['phlegm'].mean():.2f}")
242 print(f"    痰湿积分标准差: {high_risk_group['phlegm'].std():.2f}")
243 print(f"    TG均值: {high_risk_group['tg'].mean():.2f}")
244 print(f"    血脂异常风险项数均值: {high_risk_group['lipid_risk_cnt'].mean():.2f}")
245 print(f"    活动量表总分均值: {high_risk_group['mobility'].mean():.2f}")
246
247 # 年龄分布修正
248 age_dist = high_risk_group['age_group'].value_counts().sort_index()
249 print("    年龄组分布: ")
250 age_labels = {1: '40-49', 2: '50-59', 3: '60-69', 4: '70-79', 5: '80-89'}
251 for ag, cnt in age_dist.items():
252     print(f"        {age_labels[ag]}岁: {cnt}例 ({cnt/len(high_risk_group)*100:.1f}%)")
253 pct_60_79 = ((high_risk_group['age_group'] == 3) | (high_risk_group['age_group'] == 4))
254     .mean()
255 print(f"    60-79岁占比: {pct_60_79*100:.1f}%")
256 print("    结论: 高风险人群年龄分布较为分散, 并非集中于60-79岁。")
257
258 # ===== 2.8 核心特征组合识别 =====
259 print("\n【2.7】核心特征组合 (条件概率) : ")
260 # 组合1: 痰湿>=60 & 活动<40
261 combo1 = df[(df['phlegm'] >= 60) & (df['mobility'] < 40)]
262 p1 = combo1['risk_level'].value_counts(normalize=True).get('高', 0)
263 print(f"    痰湿>=60 & 活动总分<40 高风险概率: {p1:.2%} (n={len(combo1)})")
264
265 # 组合2: 痰湿>=60 & BMI>=24 & TG>1.7
266 combo2 = df[(df['phlegm'] >= 60) & (df['bmi'] >= 24) & (df['tg'] > 1.7)]
267 p2 = combo2['risk_level'].value_counts(normalize=True).get('高', 0)
268 print(f"    痰湿>=60 & BMI>=24 & TG>1.7 高风险概率: {p2:.2%} (n={len(combo2)})")
269
270 # 组合3: 痰湿为最高体质 & 血脂异常>=2项
271 constitution_cols = ['pinghe', 'qixu', 'yangxu', 'yinxu', 'phlegm', 'shire', 'xueyu', 'qiyu', 'tebing']
272 combo3 = df[(df[constitution_cols].idxmax(axis=1) == 'phlegm') & (df['lipid_risk_cnt']
273     >= 2)]
274 p3 = combo3['risk_level'].value_counts(normalize=True).get('高', 0)
275 print(f"    痰湿为最高体质 & 血脂异常>=2项 高风险概率: {p3:.2%} (n={len(combo3)})")
276
277 # 保存核心特征组合结果

```

```

276 combo_df = pd.DataFrame({
277     '特征组合': [
278         '痰湿≥60且活动总分<40',
279         '痰湿≥60且BMI≥24且TG>1.7',
280         '痰湿为最高体质且血脂异常≥2项'
281     ],
282     '高风险命中率': [p1, p2, p3],
283     '覆盖人数': [len(combo1), len(combo2), len(combo3)]
284 })
285 combo_df.to_csv('output/p2_feature_combos.csv', index=False)
286 print("\n核心特征组合已保存至output/p2_feature_combos.csv")
287
288 # 三层判定规则（指导教师B建议）
289 print("\n【2.8】三层判定规则：")
290 print("第1层（确诊/近确诊）：若TC>6.2或TG>1.7或LDL-C>3.1或HDL-C<1.04，")
291 print("直接判为'血脂异常高关注人群'。")
292 print("第2层（未病预警）：在未确诊或血脂正常者中，用体质+活动+基础信息评估风险。")
293 print("第3层（临床锚点修正）：")
294 print("血脂异常且痰湿积分>=60高风险复核")
295 print("血脂正常但痰湿积分>=80且活动能力<40高风险复核")
296
297 # ===== 2.9 权重敏感性分析 =====
298 print("\n【2.9】风险评分权重敏感性分析（固定阈值，主观权重±20%扰动）：")
299 base_weights = (alpha, beta, gamma, delta)
300 perturbations = [
301     (alpha*0.8, beta*1.2, gamma*1.2, delta), # w1-20%
302     (alpha*1.2, beta*0.8, gamma*0.8, delta), # w1+20%
303     (alpha, beta*0.8, gamma*1.2, delta), # w2-20%
304     (alpha, beta*1.2, gamma*0.8, delta), # w2+20%
305 ]
306 # 重新归一化前3项
307 base_w123 = base_weights[:3]
308 base_w123_norm = tuple(w/sum(base_w123) for w in base_w123)
309
310 base_rs = (base_w123_norm[0] * probab_prev +
311            base_w123_norm[1] * (df['phlegm'] / 100) +
312            base_w123_norm[2] * ((100 - df['mobility']) / 100) +
313            base_weights[3] * (df['constitution'] == 5).astype(int))
314 base_rl = pd.cut(base_rs, bins=[-np.inf, low_threshold, high_threshold, np.inf], labels
    =['低', '中', '高'])

```

```

315 base_dist = base_rl.value_counts(normalize=True).sort_index()
316 print(f"基准权重:低={base_dist.get('低',0):.3f},中={base_dist.get('中',0):.3f},高
    ={base_dist.get('高',0):.3f}")
317
318 for i, (w1, w2, w3, w4) in enumerate(perturbations, 1):
319     s = w1 + w2 + w3
320     w1n, w2n, w3n = w1/s, w2/s, w3/s
321     rs = w1n * probab_prev + w2n * (df['phlegm'] / 100) + w3n * ((100 - df['mobility']) /
        100) + w4 * (df['constitution'] == 5).astype(int)
322     rl = pd.cut(rs, bins=[-np.inf, low_threshold, high_threshold, np.inf], labels=['低',
        , '中', '高'])
323     dist = rl.value_counts(normalize=True).sort_index()
324     print(f"扰动{i}:低={dist.get('低',0):.3f},中={dist.get('中',0):.3f},高={dist.
        get('高',0):.3f}")
325
326 # ===== 2.10 delta 敏感性分析 =====
327 print("\n【2.10】痰湿体质标签修正系数_delta敏感性分析:")
328 print(" (delta=0.15为经验参数, 检验其对高风险比例的影响) ")
329 for d_test in [0, 0.12, 0.15, 0.18, 0.30]:
330     rs_test = (alpha * probab_prev +
331                beta * (df['phlegm'] / 100) +
332                gamma * ((100 - df['mobility']) / 100) +
333                d_test * (df['constitution'] == 5).astype(int))
334     rl_test = pd.cut(rs_test, bins=[-np.inf, low_threshold, high_threshold, np.inf],
        labels=['低', '中', '高'])
335     high_pct = (rl_test == '高').mean()
336     print(f"delta={d_test:.2f}:高风险比例={high_pct:.1%}")
337
338 # ===== 2.11 ROC曲线 (两层模型) =====
339 fpr_prev, tpr_prev, _ = roc_curve(df['disease'], probab_prev)
340
341 # ===== 保存结果 =====
342 result_df = pd.DataFrame({
343     'id': df['id'],
344     'risk_score': risk_score,
345     'risk_level': df['risk_level'],
346     'disease': df['disease'],
347     'probab_prev': probab_prev,
348     'probab_diag': probab_diag,
349     'oof_probab_prev': oof_probab_prev,

```

```

350     'oof_prob_diag': oof_prob_diag,
351     'phlegm': df['phlegm'],
352     'mobility': df['mobility'],
353     'constitution': df['constitution'],
354     'lipid_risk_cnt': df['lipid_risk_cnt']
355 })
356 result_df.to_csv('output/p2_risk_scores.csv', index=False)
357
358 # 保存熵权结果
359 entropy_df = pd.DataFrame({
360     '指标': ['P_prev', 'S_phlegm/100', '(100-M)/100'],
361     '熵权法权重': [weights_entropy[c] for c in entropy_cols],
362     '组合权重': [combined_w[c] for c in entropy_cols]
363 })
364 entropy_df.to_csv('output/p2_entropy_weights.csv', index=False)
365
366 # ===== 可视化 =====
367 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
368
369 # 图2a: 两层模型ROC对比
370 axes[0, 0].plot(fpr_prev, tpr_prev, label=f'未病预警_AUC={np.mean(auc_prev_cv):.3f}',
371                linewidth=2)
372 axes[0, 0].plot([0, 1], [0, 1], 'k--', label='Random')
373 axes[0, 0].set_xlabel('False_Positive_Rate')
374 axes[0, 0].set_ylabel('True_Positive_Rate')
375 axes[0, 0].set_title('问题2: 两层模型ROC曲线对比')
376 axes[0, 0].legend()
377
378 # 图2b: 风险评分分布
379 sns.histplot(data=df, x='risk_score', hue='risk_level', bins=30, kde=True, ax=axes[0,
380                1])
381 axes[0, 1].axvline(low_threshold, color='green', linestyle='--', label=f'低中阈值={
382                low_threshold:.3f}')
383 axes[0, 1].axvline(high_threshold, color='red', linestyle='--', label=f'中高阈值={
384                high_threshold:.3f}')
385 axes[0, 1].set_title('问题2: 综合风险评分分布')
386 axes[0, 1].legend()
387
388 # 图2c: 高风险人群年龄分布
389 age_dist_df = pd.DataFrame({'年龄组': [age_labels[i] for i in age_dist.index], '人数':

```

```

    age_dist.values})
386 sns.barplot(data=age_dist_df, x='年龄组', y='人数', ax=axes[1, 0])
387 axes[1, 0].set_title('问题2: 高风险人群年龄分布')
388
389 # 图2d: 熵权法权重对比
390 entropy_melt = entropy_df.melt(id_vars='指标', var_name='方法', value_name='权重')
391 sns.barplot(data=entropy_melt, x='指标', y='权重', hue='方法', ax=axes[1, 1])
392 axes[1, 1].set_title('问题2: 熵权法 vs 组合赋权')
393 axes[1, 1].tick_params(axis='x', rotation=15)
394
395 plt.tight_layout()
396 plt.savefig('output/fig2_problem2_risk.png', dpi=300)
397 plt.close()
398
399 print("\n 图表已保存至 output/fig2_problem2_risk.png")
400 print(" 数据已保存至 output/p2_*.csv")

```

Listing 3: problem3.py: 问题 3 干预方案优化模型

```

1  """
2  问题3: 干预方案优化模型（动态逐月规划）
3  核心升级：调理分级x不再固定为初始级别，而是每月根据当前痰湿积分动态调整
4  包含：逐月模拟、样本1/2/3月度轨迹、全体278位患者优化、依从性衰减敏感性分析
5  """
6  import pandas as pd
7  import numpy as np
8  import matplotlib
9  matplotlib.use('Agg')
10 import matplotlib.pyplot as plt
11 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS']
12 plt.rcParams['axes.unicode_minus'] = False
13 import seaborn as sns
14 import os
15 import warnings
16 warnings.filterwarnings('ignore')
17
18 np.random.seed(42)
19 os.makedirs('output', exist_ok=True)
20
21 # ===== 数据读取 =====

```



```

22 df = pd.read_csv('题目（处理后）/附件1: 样例数据.csv')
23
24 col_map = {
25     '痰湿质': 'phlegm',
26     '平和质': 'pinghe', '气虚质': 'qixu', '阳虚质': 'yangxu', '阴虚质': 'yinxu',
27     '湿热质': 'shire', '血瘀质': 'xueyu', '气郁质': 'qiyu', '特禀质': 'tebing',
28     'ADL总分': 'adl', 'IADL总分': 'iadl',
29     '活动量表总分（ADL总分+IADL总分）': 'mobility',
30     'HDL-C（高密度脂蛋白）': 'hdl', 'LDL-C（低密度脂蛋白）': 'ldl',
31     'TG（甘油三酯）': 'tg', 'TC（总胆固醇）': 'tc',
32     '空腹血糖': 'glucose', '尿酸': 'uric', 'BMI': 'bmi',
33     '高血脂症二分类标签': 'disease',
34     '血脂异常分型标签（确诊病例）': 'disease_type',
35     '年龄组': 'age_group', '性别': 'gender',
36     '吸烟史': 'smoke', '饮酒史': 'drink',
37     '样本ID': 'id', '体质标签': 'constitution'
38 }
39 df.rename(columns=col_map, inplace=True)
40
41 print("=" * 60)
42 print("问题3: 干预方案优化模型（动态逐月规划）")
43 print("=" * 60)
44
45 # ===== 参数设定 =====
46 C_TREAT = {1: 30, 2: 80, 3: 130}
47 C_ACT = {1: 3, 2: 5, 3: 8}
48 WEEKS_PER_MONTH = 4
49 MONTHS = 6
50 MAX_COST = 2000
51
52 def get_treatment_level(phlegm_score):
53     if phlegm_score <= 58:
54         return 1
55     elif phlegm_score <= 61:
56         return 2
57     else:
58         return 3
59
60 def get_max_activity_intensity(age_group, mobility_score):
61     if age_group in [1, 2]:

```

```

62     y_max_age = 3
63 elif age_group in [3, 4]:
64     y_max_age = 2
65 else:
66     y_max_age = 1
67 if mobility_score < 40:
68     y_max_mob = 1
69 elif mobility_score < 60:
70     y_max_mob = 2
71 else:
72     y_max_mob = 3
73 return min(y_max_age, y_max_mob)
74
75 def calc_monthly_drop_rate(y, f, age_group, mobility, compliance_params=None):
76     """
77     计算每月痰湿积分下降率（含依从性衰减）
78     compliance_params: 可传入自定义衰减参数做敏感性分析
79     默认: {'age5_factor': 0.5, 'age5_cap': 6.0, 'mob_low_factor': 0.6, 'mob_low_cap':
80           6.2, 'age34_factor': 0.8}
81     """
82     if compliance_params is None:
83         compliance_params = {
84             'age5_factor': 0.5, 'age5_cap': 6.0,
85             'mob_low_factor': 0.6, 'mob_low_cap': 6.2,
86             'age34_factor': 0.8
87         }
88     if f < 5:
89         return 0.0
90     if age_group == 5:
91         effective_f = min(5 + (f - 5) * compliance_params['age5_factor'],
92                           compliance_params['age5_cap'])
93     elif mobility < 40:
94         effective_f = min(5 + (f - 5) * compliance_params['mob_low_factor'],
95                           compliance_params['mob_low_cap'])
96     elif age_group in [3, 4]:
97         effective_f = 5 + (f - 5) * compliance_params['age34_factor']
98     else:
99         effective_f = f
100     base_drop = 0.03 * (y - 1)
101     extra_drop = 0.01 * (effective_f - 5)

```

```

99     return base_drop + extra_drop
100
101 def simulate_months(s0, y, f, age_group, mobility, months=MONTHS, compliance_params=
    None):
102     """
103     逐月模拟干预过程，调理分级x每月根据当前积分动态调整
104     返回：{
105         's_trajectory': [S0, S1, S2, S3, S4, S5, S6] (每月初的积分，S6为6个月末)，
106         'x_trajectory': [x1, x2, x3, x4, x5, x6] (每月的调理分级)，
107         'cost_trajectory': [c1, c2, c3, c4, c5, c6] (每月成本)，
108         'total_cost': 总成本，
109         's_final': 最终积分
110     }
111     """
112     s_current = s0
113     s_trajectory = [s_current]
114     x_trajectory = []
115     cost_trajectory = []
116     total_cost = 0
117     r = calc_monthly_drop_rate(y, f, age_group, mobility, compliance_params)
118
119     for _ in range(months):
120         x = get_treatment_level(s_current)
121         monthly_cost = C_TREAT[x] + WEEKS_PER_MONTH * f * C_ACT[y]
122         total_cost += monthly_cost
123         if total_cost > MAX_COST:
124             return None # 超支，非法方案
125         x_trajectory.append(x)
126         cost_trajectory.append(monthly_cost)
127         s_current = s_current * (1 - r)
128         s_trajectory.append(s_current)
129
130     return {
131         's_trajectory': s_trajectory,
132         'x_trajectory': x_trajectory,
133         'cost_trajectory': cost_trajectory,
134         'total_cost': total_cost,
135         's_final': s_current
136     }
137

```

```

138 def optimize_intervention_dynamic(patient, compliance_params=None):
139     """
140     动态逐月规划优化：遍历所有合法(y, f)，逐月模拟，选择最终积分最小者
141     """
142     s0 = patient['phlegm']
143     y_max = get_max_activity_intensity(patient['age_group'], patient['mobility'])
144
145     best_plan = None
146     best_final = float('inf')
147
148     for y in range(1, y_max + 1):
149         for f in range(1, 11):
150             result = simulate_months(s0, y, f, patient['age_group'], patient['mobility'],
151                                     compliance_params=compliance_params)
152             if result is None:
153                 continue
154             if result['s_final'] < best_final - 1e-6:
155                 best_final = result['s_final']
156                 best_plan = {
157                     'id': patient['id'],
158                     'y_level': y,
159                     'freq': f,
160                     's0': s0,
161                     **result
162                 }
163             elif abs(result['s_final'] - best_final) < 1e-6 and result['total_cost'] <
164                 best_plan['total_cost']:
165                 best_plan['total_cost'] = result['total_cost']
166                 best_plan['y_level'] = y
167                 best_plan['freq'] = f
168                 best_plan['total_cost'] = result['total_cost']
169                 best_plan['s_trajectory'] = result['s_trajectory']
170                 best_plan['x_trajectory'] = result['x_trajectory']
171                 best_plan['cost_trajectory'] = result['cost_trajectory']
172                 best_plan['s_final'] = result['s_final']
173
174     return best_plan
175
176 # ===== 样本ID 1、2、3月度轨迹输出 =====
177 print("\n 【1】 样本ID_1、2、3的最优干预方案（动态逐月规划）：\n")
178 sample_ids = [1, 2, 3]

```

```

176 plans_123 = []
177 for sid in sample_ids:
178     p = df[df['id'] == sid].iloc[0]
179     if p['constitution'] != 5:
180         print(f"⚠️注意：样本ID_{sid}不是痰湿体质（constitution={int(p['constitution'])}），仍按题目要求输出方案。\\n")
181     plan = optimize_intervention_dynamic(p)
182     plans_123.append(plan)
183     lvl_name = ('基础', '中度', '强化')
184     print(f"样本ID_{int(plan['id'])}：⚠️痰湿积分={plan['s0']}，⚠️年龄组={int(p['age_group'])}，⚠️活动总分={p['mobility']}")
185     print(f"⚠️活动强度：{plan['y_level']}级（单次{C_ACT[plan['y_level']] }元），⚠️每周频率：{plan['freq']}次")
186     print(f"⚠️6个月总成本：{plan['total_cost']}元，⚠️预期6个月后痰湿积分：{plan['s_final']:.2f}（下降{plan['s0']-plan['s_final']:.2f}分）")
187     print(f"⚠️{'月份':>4}{ '月初积分':>8}{ '调理分级':>8}{ '月成本(元)':>10}")
188     for m in range(MONTHS):
189         print(f"⚠️{m+1:>4}{plan['s_trajectory'][m]:>8.2f}{lvl_name[plan['x_trajectory'][m]-1]+'('+str(plan['x_trajectory'][m])+')':>8}{plan['cost_trajectory'][m]:>10}")
190     print(f"⚠️{'期末':>4}{plan['s_trajectory'][MONTHS]:>8.2f}")
191     print()
192
193 # ===== 全体痰湿体质患者优化（n=278） =====
194 print("【2】对全体痰湿体质患者（体质标签=5）执行优化，n=278")
195 phlegm_patients = df[df['constitution'] == 5].copy()
196 print(f"⚠️确认：痰湿体质患者人数={len(phlegm_patients)}")
197
198 all_plans = []
199 for _, p in phlegm_patients.iterrows():
200     all_plans.append(optimize_intervention_dynamic(p))
201
202 # 过滤掉None（理论上不应出现，因为至少存在低成本方案）
203 all_plans = [p for p in all_plans if p is not None]
204 plan_df = pd.DataFrame(all_plans)
205
206 phlegm_patients['opt_y'] = plan_df['y_level'].values
207 phlegm_patients['opt_f'] = plan_df['freq'].values
208 phlegm_patients['opt_cost'] = plan_df['total_cost'].values
209 phlegm_patients['opt_s_final'] = plan_df['s_final'].values

```

```

210 phlegm_patients['opt_s_drop'] = plan_df['s0'].values - plan_df['s_final'].values
211
212 print("\n【3】患者特征—最优方案匹配规律：")
213
214 # 按年龄组
215 print("\n(a) 按年龄组统计最优方案均值：")
216 age_summary = phlegm_patients.groupby('age_group').agg({
217     'opt_y': 'mean',
218     'opt_f': 'mean',
219     'opt_cost': 'mean',
220     'opt_s_final': 'mean',
221     'opt_s_drop': 'mean'
222 }).round(2)
223 print(age_summary)
224
225 # 按活动能力分组
226 print("\n(b) 按活动能力分组统计最优方案均值：")
227 phlegm_patients['mobility_group'] = pd.cut(
228     phlegm_patients['mobility'],
229     bins=[0, 40, 60, 100],
230     labels=['差(<40)', '中(40-60)', '好(≥60)'],
231     include_lowest=True
232 )
233 mob_summary = phlegm_patients.groupby('mobility_group').agg({
234     'opt_y': 'mean',
235     'opt_f': 'mean',
236     'opt_cost': 'mean',
237     'opt_s_final': 'mean',
238     'opt_s_drop': 'mean'
239 }).round(2)
240 print(mob_summary)
241
242 # 按痰湿积分分组
243 print("\n(c) 按痰湿积分分组统计最优方案均值：")
244 phlegm_patients['phlegm_group'] = pd.cut(
245     phlegm_patients['phlegm'],
246     bins=[0, 58, 61, 100],
247     labels=['低(0-58)', '中(59-61)', '高(≥62)'],
248     include_lowest=True
249 )

```

```

250 phlegm_summary = phlegm_patients.groupby('phlegm_group').agg({
251     'opt_y': 'mean',
252     'opt_f': 'mean',
253     'opt_cost': 'mean',
254     'opt_s_final': 'mean',
255     'opt_s_drop': 'mean'
256 }).round(2)
257 print(phlegm_summary)
258
259 # 成本-效果分析
260 print("\n 【4】 单位成本降分效果（按初始调理分级统计）：")
261 # 注意：由于动态调整，患者可能在干预过程中改变调理分级
262 # 这里按"初始调理分级"（基于初始积分）统计
263 phlegm_patients['initial_x'] = phlegm_patients['phlegm'].apply(get_treatment_level)
264 cost_effect = phlegm_patients.groupby('initial_x').agg({
265     'opt_s_drop': 'mean',
266     'opt_cost': 'mean'
267 }).round(2)
268 cost_effect['单位成本降分(分/元)'] = (cost_effect['opt_s_drop'] / cost_effect['opt_cost
    ']).round(4)
269 cost_effect.index = ['基础(x=1)', '中度(x=2)', '强化(x=3)']
270 print(cost_effect)
271
272 # ===== 依从性衰减敏感性分析 =====
273 print("\n 【5】 依从性衰减敏感性分析：")
274 sensitivity_configs = {
275     '保守方案': {'age5_factor': 0.3, 'age5_cap': 5.5, 'mob_low_factor': 0.4, '
        mob_low_cap': 5.5, 'age34_factor': 0.6},
276     '基准方案': {'age5_factor': 0.5, 'age5_cap': 6.0, 'mob_low_factor': 0.6, '
        mob_low_cap': 6.2, 'age34_factor': 0.8},
277     '乐观方案': {'age5_factor': 0.7, 'age5_cap': 7.0, 'mob_low_factor': 0.8, '
        mob_low_cap': 7.5, 'age34_factor': 0.9},
278 }
279
280 sens_results = []
281 for config_name, params in sensitivity_configs.items():
282     sid_results = []
283     for sid in sample_ids:
284         p = df[df['id'] == sid].iloc[0]
285         plan = optimize_intervention_dynamic(p, compliance_params=params)

```

```

286         sid_results.append({
287             'config': config_name,
288             'id': sid,
289             'freq': plan['freq'],
290             'cost': plan['total_cost'],
291             's_drop': plan['s0'] - plan['s_final']
292         })
293     # 计算该配置下278人的均值
294     temp_plans = []
295     for _, p in phlegm_patients.iterrows():
296         temp_plans.append(optimize_intervention_dynamic(p, compliance_params=params))
297     temp_plans = [p for p in temp_plans if p is not None]
298     avg_cost = np.mean([p['total_cost'] for p in temp_plans])
299     avg_drop = np.mean([p['s0'] - p['s_final'] for p in temp_plans])
300     for sr in sid_results:
301         sr['avg_cost_278'] = avg_cost
302         sr['avg_drop_278'] = avg_drop
303         sens_results.append(sr)
304
305 sens_df = pd.DataFrame(sens_results)
306 print("\n敏感性分析结果:")
307 for config_name in sensitivity_configs:
308     sub = sens_df[sens_df['config'] == config_name]
309     print(f"\n{config_name}:")
310     print(f"278人平均成本:{sub['avg_cost_278'].iloc[0]:.2f}元,平均降分:{sub['avg_drop_278'].iloc[0]:.2f}分")
311     for _, row in sub.iterrows():
312         print(f"ID{int(row['id'])}: 最优频率={int(row['freq'])}次/周, 成本={row['cost']:.0f}元, 降分={row['s_drop']:.2f}分")
313
314 # ===== 保存结果 =====
315 phlegm_patients.to_csv('output/p3_phlegm_patients_results.csv', index=False)
316 sens_df.to_csv('output/p3_sensitivity_analysis.csv', index=False)
317
318 # 保存样本1,2,3的月度轨迹
319 for plan in plans_123:
320     traj_df = pd.DataFrame({
321         '月份': list(range(1, MONTHS+1)) + ['期末'],
322         '月初积分': plan['s_trajectory'],
323         '调理分级': plan['x_trajectory'] + ['--'],

```



```

324         '月成本(元)': plan['cost_trajectory'] + [plan['total_cost']]
325     })
326     traj_df.to_csv(f"output/p3_id{int(plan['id'])}_trajectory.csv", index=False)
327
328     # ===== 可视化 =====
329     fig, axes = plt.subplots(2, 2, figsize=(16, 12))
330
331     # 图3a: 样本1,2,3的干预历程折线图
332     for plan in plans_123:
333         months_axis = list(range(MONTHS + 1))
334         axes[0, 0].plot(months_axis, plan['s_trajectory'], marker='o', label=f"ID_{int(plan
335             ['id'])}")
336     axes[0, 0].set_xlabel('月份')
337     axes[0, 0].set_ylabel('痰湿积分')
338     axes[0, 0].set_title('问题3: 样本ID_1/2/3_干预历程 (痰湿积分变化) ')
339     axes[0, 0].legend()
340     axes[0, 0].set_xticks(range(MONTHS + 1))
341     axes[0, 0].set_xticklabels(['初始'] + [f'{m}月' for m in range(1, MONTHS+1)])
342     axes[0, 0].grid(True, alpha=0.3)
343
344     # 图3b: 样本1,2,3的月度成本柱状图
345     width = 0.25
346     x_pos = np.arange(MONTHS)
347     for i, plan in enumerate(plans_123):
348         axes[0, 1].bar(x_pos + i*width, plan['cost_trajectory'], width, label=f"ID_{int(
349             plan['id'])}")
350     axes[0, 1].set_xlabel('月份')
351     axes[0, 1].set_ylabel('月度成本 (元) ')
352     axes[0, 1].set_title('问题3: 样本ID_1/2/3_月度成本分布')
353     axes[0, 1].set_xticks(x_pos + width)
354     axes[0, 1].set_xticklabels([f'{m}月' for m in range(1, MONTHS+1)])
355     axes[0, 1].legend()
356     axes[0, 1].grid(True, alpha=0.3)
357
358     # 图3c: 最优频率分布 (按年龄组)
359     sns.boxplot(data=phlegm_patients, x='age_group', y='opt_f', ax=axes[1, 0])
360     axes[1, 0].set_title('问题3: 最优训练频率按年龄组分布 (n=278) ')
361     axes[1, 0].set_xlabel('年龄组')
362     axes[1, 0].set_ylabel('最优频率 (次/周) ')

```

```

362 # 图3d: 敏感性分析对比
363 sens_pivot = sens_df.pivot_table(index='config', values=['avg_cost_278', 'avg_drop_278'
    ], aggfunc='first')
364 sens_pivot = sens_pivot.reindex(['保守方案', '基准方案', '乐观方案'])
365 x_labels = sens_pivot.index
366 x_pos = np.arange(len(x_labels))
367 ax_twin = axes[1, 1].twinx()
368 bars = axes[1, 1].bar(x_pos - 0.2, sens_pivot['avg_cost_278'], 0.4, label='平均成本',
    color='steelblue')
369 line = ax_twin.plot(x_pos + 0.2, sens_pivot['avg_drop_278'], 'ro-', label='平均降分',
    linewidth=2, markersize=8)
370 axes[1, 1].set_xlabel('依从性衰减假设')
371 axes[1, 1].set_ylabel('平均成本 (元)', color='steelblue')
372 ax_twin.set_ylabel('平均降分 (分)', color='red')
373 axes[1, 1].set_title('问题3: 依从性衰减敏感性分析 (278人平均)')
374 axes[1, 1].set_xticks(x_pos)
375 axes[1, 1].set_xticklabels(x_labels)
376 axes[1, 1].legend(loc='upper_left')
377 ax_twin.legend(loc='upper_right')
378
379 plt.tight_layout()
380 plt.savefig('output/fig3_problem3_optimization.png', dpi=300)
381 plt.close()
382
383 print("\n 图表已保存至 output/fig3_problem3_optimization.png")
384 print(" 数据已保存至 output/p3_*.csv")

```

Listing 4: figures.py: 综合制图程序

```

1  """
2  制图专用程序: 生成论文所需的最终高质量图表
3  读取 problem1/2/3 的输出数据, 绘制综合可视化图
4  """
5  import pandas as pd
6  import numpy as np
7  import matplotlib
8  matplotlib.use('Agg')
9  import matplotlib.pyplot as plt
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS']
11 plt.rcParams['axes.unicode_minus'] = False

```

```

12 import seaborn as sns
13 import os
14
15 os.makedirs('output', exist_ok=True)
16
17 # 设置全局样式
18 sns.set_style("whitegrid")
19 plt.rcParams['figure.dpi'] = 300
20
21 print("=" * 60)
22 print("制图专用程序：生成论文最终图表")
23 print("=" * 60)
24
25 # ===== 图1：问题1综合图 =====
26 print("\n【图1】问题1：特征筛选与体质贡献度综合图")
27
28 fig = plt.figure(figsize=(18, 10))
29 # 使用GridSpec进行复杂布局
30 from matplotlib.gridspec import GridSpec
31 gs = GridSpec(2, 3, figure=fig, hspace=0.35, wspace=0.3)
32
33 # 1a: 指标相关性对比
34 corr_data = pd.read_csv('output/p1_correlation.csv')
35 ax1 = fig.add_subplot(gs[0, 0])
36 corr_melt = corr_data[['指标', '与痰湿质Pearson', '与高血脂点二列']].melt(
37     id_vars='指标', var_name='目标', value_name='r')
38 sns.barplot(data=corr_melt, x='指标', y='r', hue='目标', ax=ax1, palette='Set2')
39 ax1.set_title('(a) 关键指标与目标变量的相关性')
40 ax1.tick_params(axis='x', rotation=45)
41 ax1.axhline(0, color='black', linewidth=0.5)
42
43 # 1b: 九种体质Logistic系数
44 coef_df = pd.read_csv('output/p1_constitution_logistic.csv')
45 ax2 = fig.add_subplot(gs[0, 1])
46 coef_plot = coef_df.sort_values('abs_coef', ascending=False)
47 colors = ['red' if p >= 0.05 else 'steelblue' for p in coef_plot['pvalue']]
48 sns.barplot(data=coef_plot, x='体质', y='coef', palette=colors, ax=ax2)
49 ax2.set_title('(b) 九种体质标准化Logistic系数')
50 ax2.tick_params(axis='x', rotation=45)
51 ax2.axhline(0, color='black', linewidth=0.5)

```

```

52 # 添加显著性说明
53 from matplotlib.patches import Patch
54 legend_elements = [Patch(facecolor='red', label='p≥0.05 (不显著)'),
55                    Patch(facecolor='steelblue', label='p<0.05')]
56 ax2.legend(handles=legend_elements, loc='upper_right', fontsize=8)
57
58 # 1c: 随机森林特征重要性
59 rf_imp = pd.read_csv('output/p1_rf_importance.csv')
60 ax3 = fig.add_subplot(gs[0, 2])
61 sns.barplot(data=rf_imp, x='体质', y='Gini重要性', ax=ax3, palette='viridis')
62 ax3.set_title('(c) 随机森林Gini特征重要性')
63 ax3.tick_params(axis='x', rotation=45)
64
65 # 1d: VIF诊断
66 vif_data = pd.read_csv('output/p1_vif.csv')
67 ax4 = fig.add_subplot(gs[1, 0])
68 sns.barplot(data=vif_data, x='体质', y='VIF', ax=ax4, palette='coolwarm')
69 ax4.set_title('(d) 九种体质VIF值')
70 ax4.tick_params(axis='x', rotation=45)
71 ax4.axhline(5, color='red', linestyle='--', linewidth=1.5, label='VIF=5阈值')
72 ax4.legend()
73
74 # 1e: 组间差异检验效应量（从problem1输出中读取需要的数据，这里简化展示）
75 ax5 = fig.add_subplot(gs[1, 1])
76 ax5.text(0.5, 0.5, '问题1结论: \n\n'
77         '1. 血脂指标(TG/TC/LDL-C/HDL-C)与尿酸对高血脂标签有显著预警能力\n'
78         '但与痰湿质积分的线性关联极弱。 \n\n'
79         '2. 关键指标分为两类: \n'
80         '· 高脂风险识别指标: TG、TC、LDL-C、HDL-C、尿酸\n'
81         '· 痰湿严重程度直接表征指标: 痰湿质积分、主体质标签\n\n'
82         '3. 九种体质标准化系数均未达显著水平(p≥0.12), \n'
83         '随机森林显示痰湿质Gini重要性相对较高, 提示非线性交互效应存在。 \n\n'
84         '4. VIF均低于5, 多重共线性不是体质系数不显著的主因。',
85         transform=ax5.transAxes, fontsize=11, verticalalignment='center',
86         horizontalalignment='center', bbox=dict(boxstyle='round', facecolor='wheat',
87         alpha=0.3))
87 ax5.set_xlim(0, 1)
88 ax5.set_ylim(0, 1)
89 ax5.axis('off')
90 ax5.set_title('(e) 问题1核心结论')

```

```

91
92 plt.savefig('output/fig1_final_problem1.png', dpi=300, bbox_inches='tight')
93 plt.close()
94 print("\n已保存output/fig1_final_problem1.png")
95
96 # ===== 图2: 问题2综合图 =====
97 print("\n【图2】问题2: 风险预警模型综合图")
98
99 fig = plt.figure(figsize=(18, 10))
100 gs = GridSpec(2, 3, figure=fig, hspace=0.35, wspace=0.3)
101
102 # 2a: 风险评分分布
103 result_df = pd.read_csv('output/p2_risk_scores.csv')
104 ax1 = fig.add_subplot(gs[0, 0])
105 sns.histplot(data=result_df, x='risk_score', hue='risk_level', bins=30, kde=True, ax=
    ax1, palette='RdYlGn_r')
106 q30 = result_df['risk_score'].quantile(0.30)
107 q90 = result_df['risk_score'].quantile(0.90)
108 ax1.axvline(q30, color='green', linestyle='--', linewidth=1.5, label=f'低中阈值={q30:.3
    f}')
109 ax1.axvline(q90, color='red', linestyle='--', linewidth=1.5, label=f'中高阈值={q90:.3f
    }')
110 ax1.set_title('(a)综合风险评分分布')
111 ax1.legend(fontsize=8)
112
113 # 2b: 两层模型ROC对比 (使用out-of-fold预测概率)
114 ax2 = fig.add_subplot(gs[0, 1])
115 from sklearn.metrics import roc_curve
116 fpr_prev, tpr_prev, _ = roc_curve(result_df['disease'], result_df['oof_prob_prev'])
117 auc_prev = np.trapezoid(tpr_prev, fpr_prev)
118 ax2.plot(fpr_prev, tpr_prev, label=f'未病预警AUC={auc_prev:.3f}', linewidth=2, color='
    steelblue')
119 ax2.plot([0, 1], [0, 1], 'k--', label='Random', linewidth=1)
120 ax2.set_xlabel('False Positive Rate')
121 ax2.set_ylabel('True Positive Rate')
122 ax2.set_title('(b)未病预警模型out-of-fold ROC曲线')
123 ax2.legend()
124
125 # 2c: 高风险人群年龄分布
126 ax3 = fig.add_subplot(gs[0, 2])

```

```

127 high_risk = result_df[result_df['risk_level'] == '高']
128 age_dist = high_risk['constitution'].value_counts() # 这里需要重新读取年龄
129 # 由于result_df没有年龄，我们用原始数据
130 df_raw = pd.read_csv('题目（处理后）/附件1: 样例数据.csv')
131 df_raw['risk_level'] = result_df['risk_level'].values
132 high_risk_full = df_raw[df_raw['risk_level'] == '高']
133 age_dist = high_risk_full['年龄组'].value_counts().sort_index()
134 age_labels = {1: '40-49', 2: '50-59', 3: '60-69', 4: '70-79', 5: '80-89'}
135 age_dist_df = pd.DataFrame({'年龄组': [age_labels[i] for i in age_dist.index], '人数':
    age_dist.values})
136 sns.barplot(data=age_dist_df, x='年龄组', y='人数', ax=ax3, palette='Blues_d')
137 ax3.set_title(f'(c)高风险人群年龄分布(n={len(high_risk)})')
138
139 # 2d: 熵权法权重
140 ax4 = fig.add_subplot(gs[1, 0])
141 entropy_df = pd.read_csv('output/p2_entropy_weights.csv')
142 entropy_melt = entropy_df.melt(id_vars='指标', var_name='方法', value_name='权重')
143 sns.barplot(data=entropy_melt, x='指标', y='权重', hue='方法', ax=ax4, palette='Set1')
144 ax4.set_title('(d)熵权法vs组合赋权')
145 ax4.tick_params(axis='x', rotation=15)
146
147 # 2e: 核心特征组合条件概率
148 ax5 = fig.add_subplot(gs[1, 1:])
149 combo_data = pd.read_csv('output/p2_feature_combos.csv')
150 combo_data['特征组合'] = ['痰湿≥60\n&活动<40', '痰湿≥60\n&BMI≥24\n&TG>1.7', '痰湿为最高
    体质\n&血脂异常≥2项']
151 bars = sns.barplot(data=combo_data, x='特征组合', y='高风险命中率', ax=ax5, palette='
    Reds')
152 ax5.set_title('(e)核心特征组合模型高风险分层命中率')
153 ax5.set_ylim(0, 1)
154 for i, (p, n) in enumerate(zip(combo_data['高风险命中率'], combo_data['覆盖人数'])):
155     ax5.text(i, p + 0.02, f'{p:.1%}\n(n={n})', ha='center', fontsize=9)
156
157 plt.savefig('output/fig2_final_problem2.png', dpi=300, bbox_inches='tight')
158 plt.close()
159 print("\n已保存output/fig2_final_problem2.png")
160
161 # ===== 图3: 问题3综合图 =====
162 print("\n【图3】问题3: 干预优化综合图")
163

```

```

164 fig = plt.figure(figsize=(18, 10))
165 gs = GridSpec(2, 3, figure=fig, hspace=0.35, wspace=0.3)
166
167 # 读取样本轨迹
168 traj1 = pd.read_csv('output/p3_id1_trajectory.csv')
169 traj2 = pd.read_csv('output/p3_id2_trajectory.csv')
170 traj3 = pd.read_csv('output/p3_id3_trajectory.csv')
171
172 # 3a: 三个样本的积分变化轨迹
173 ax1 = fig.add_subplot(gs[0, :2])
174 for traj, label, color in zip([traj1, traj2, traj3],
175                               ['ID1', 'ID2', 'ID3'],
176                               ['#e74c3c', '#3498db', '#2ecc71']):
177     months = list(range(len(traj)))
178     ax1.plot(months, traj['月初积分'], marker='o', label=label, color=color, linewidth
179              =2, markersize=8)
180     # 标注调理分级变化点
181     for i in range(len(traj)-1):
182         if traj['调理分级'].iloc[i] != traj['调理分级'].iloc[i+1] and traj['调理分级'].
183             iloc[i+1] != '---':
184             ax1.axvline(i+0.5, color=color, linestyle='---', alpha=0.3)
185 ax1.set_xlabel('月份')
186 ax1.set_ylabel('痰湿积分')
187 ax1.set_title('(a) 样本ID1/2/3干预历程（痰湿积分动态变化）')
188 ax1.set_xticks(range(7))
189 ax1.set_xticklabels(['初始'] + [f'{m}月' for m in range(1, 7)])
190 ax1.legend()
191 ax1.grid(True, alpha=0.3)
192
193 # 3b: 月度成本堆叠图
194 ax2 = fig.add_subplot(gs[0, 2])
195 cost_matrix = np.array([traj1['月成本(元)'].iloc[:6].values,
196                        traj2['月成本(元)'].iloc[:6].values,
197                        traj3['月成本(元)'].iloc[:6].values])
198 x_months = np.arange(6) + 1
199 width = 0.25
200 for i, (costs, label, color) in enumerate(zip(cost_matrix, ['ID1', 'ID2', 'ID3'],
201                                                  ['#e74c3c', '#3498db', '#2ecc71'])):
202     ax2.bar(x_months + (i-1)*width, costs, width, label=label, color=color, alpha=0.8)
203 ax2.set_xlabel('月份')

```

```

202 ax2.set_ylabel('月度成本 (元) ')
203 ax2.set_title('(b) 月度成本对比')
204 ax2.set_xticks(x_months)
205 ax2.legend()
206
207 # 3c: 278人最优频率分布
208 ax3 = fig.add_subplot(gs[1, 0])
209 p3_results = pd.read_csv('output/p3_phlegm_patients_results.csv')
210 sns.boxplot(data=p3_results, x='age_group', y='opt_f', ax=ax3, palette='Set3')
211 ax3.set_title('(c) 最优频率按年龄组分布 (n=278) ')
212 ax3.set_xlabel('年龄组')
213 ax3.set_ylabel('最优频率 (次/周) ')
214
215 # 3d: 成本-效果散点图
216 ax4 = fig.add_subplot(gs[1, 1])
217 scatter = ax4.scatter(p3_results['opt_cost'], p3_results['opt_s_drop'],
218                       c=p3_results['phlegm'], cmap='RdYlGn_r', alpha=0.6, s=30)
219 ax4.set_xlabel('总成本 (元) ')
220 ax4.set_ylabel('痰湿积分下降 (分) ')
221 ax4.set_title('(d) 成本-效果散点图 (颜色=初始积分) ')
222 plt.colorbar(scatter, ax=ax4, label='初始痰湿积分')
223
224 # 3e: 敏感性分析
225 ax5 = fig.add_subplot(gs[1, 2])
226 sens_df = pd.read_csv('output/p3_sensitivity_analysis.csv')
227 sens_summary = sens_df.groupby('config').agg({
228     'avg_cost_278': 'first',
229     'avg_drop_278': 'first'
230 }).reindex(['保守方案', '基准方案', '乐观方案'])
231 x_pos = np.arange(len(sens_summary))
232 ax5_twin = ax5.twinx()
233 bars = ax5.bar(x_pos - 0.2, sens_summary['avg_cost_278'], 0.4, label='平均成本', color=
    'steelblue', alpha=0.8)
234 line = ax5_twin.plot(x_pos + 0.2, sens_summary['avg_drop_278'], 'ro-', label='平均降分'
    , linewidth=2, markersize=8)
235 ax5.set_xlabel('依从性衰减假设')
236 ax5.set_ylabel('平均成本 (元) ', color='steelblue')
237 ax5_twin.set_ylabel('平均降分 (分) ', color='red')
238 ax5.set_title('(e) 依从性衰减敏感性分析')
239 ax5.set_xticks(x_pos)

```



```

240 ax5.set_xticklabels(sens_summary.index, rotation=15)
241 ax5.legend(loc='upper_left')
242 ax5_twin.legend(loc='upper_right')
243
244 plt.savefig('output/fig3_final_problem3.png', dpi=300, bbox_inches='tight')
245 plt.close()
246 print("\n已保存_output/fig3_final_problem3.png")
247
248 print("\n" + "=" * 60)
249 print("全部图表生成完毕")
250 print("=" * 60)

```

Listing 5: fig1a_indicator_correlation.py: 图 1(a) 相关性对比

```

1  """
2  图1(a): 关键指标与痰湿质积分、高血脂标签的相关性对比条形图
3  """
4  import pandas as pd
5  import matplotlib
6  matplotlib.use('Agg')
7  import matplotlib.pyplot as plt
8  import numpy as np
9
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
11 plt.rcParams['axes.unicode_minus'] = False
12
13 df = pd.read_csv('output/p1_correlation.csv')
14
15 fig, ax = plt.subplots(figsize=(10, 6), dpi=300)
16
17 indicators = df['指标'].values
18 x = np.arange(len(indicators))
19 width = 0.35
20
21 bars1 = ax.bar(x - width/2, df['与痰湿质Pearson'], width, label='与痰湿质Pearson$r$',
22               color='steelblue', edgecolor='black', linewidth=0.5)
23 bars2 = ax.bar(x + width/2, df['与高血脂点二列'], width, label='与高血脂点二列$r$',
24               color='coral', edgecolor='black', linewidth=0.5)
25
26 ax.set_ylabel('相关系数', fontsize=12)

```

```

25 ax.set_title('关键指标与目标变量的相关性对比', fontsize=14, fontweight='bold')
26 ax.set_xticks(x)
27 ax.set_xticklabels(indicators, fontsize=10)
28 ax.axhline(0, color='black', linewidth=0.8)
29 ax.legend(fontsize=10, loc='upper_right')
30 ax.set_ylim(-0.6, 0.6)
31
32 for bar in bars2:
33     height = bar.get_height()
34     if abs(height) > 0.15:
35         ax.annotate(f'{height:.3f}',
36                     xy=(bar.get_x() + bar.get_width() / 2, height),
37                     xytext=(0, 3 if height > 0 else -12),
38                     textcoords="offset_points",
39                     ha='center', va='bottom' if height > 0 else 'top',
40                     fontsize=8, color='darkred')
41
42 plt.tight_layout()
43 plt.savefig('figures/fig1a_indicator_correlation.png', dpi=300, bbox_inches='tight')
44 print("已保存figures/fig1a_indicator_correlation.png")

```

Listing 6: fig1b_constitution_contribution.py: 图 1(b) 体质贡献度对比

```

1  """
2  图1(b): 九种体质对高血脂发病风险的贡献度对比
3  (标准化Logistic回归系数vs随机森林Gini重要性)
4  """
5  import pandas as pd
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  import numpy as np
10
11  plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
12  plt.rcParams['axes.unicode_minus'] = False
13
14  coef_df = pd.read_csv('output/p1_constitution_logistic.csv')
15  rf_df = pd.read_csv('output/p1_rf_importance.csv')
16
17  # 统一体质顺序

```

```

18 constitution_names = {
19     'pinghe': '平和质', 'qixu': '气虚质', 'yangxu': '阳虚质',
20     'yinxu': '阴虚质', 'phlegm': '痰湿质', 'shire': '湿热质',
21     'xueyu': '血瘀质', 'qiyu': '气郁质', 'tebing': '特禀质'
22 }
23
24 coef_df['体质中文'] = coef_df['体质'].map(constitution_names)
25 rf_df['体质中文'] = rf_df['体质'].map(constitution_names)
26
27 # 按Logistic系数绝对值排序
28 order = coef_df.sort_values('abs_coef', ascending=False)['体质中文'].values
29 coef_df = coef_df.set_index('体质中文').reindex(order).reset_index()
30 rf_df = rf_df.set_index('体质中文').reindex(order).reset_index()
31
32 fig, ax1 = plt.subplots(figsize=(11, 6), dpi=300)
33
34 x = np.arange(len(order))
35 width = 0.35
36
37 colors_coef = ['firebrick' if p >= 0.05 else 'steelblue' for p in coef_df['pvalue']]
38 bars1 = ax1.bar(x - width/2, coef_df['coef'], width, label='标准化Logistic系数', color=
    colors_coef, edgecolor='black', linewidth=0.5)
39 ax1.set_ylabel(r'标准化Logistic系数_{$\beta^{*}$}', color='steelblue', fontsize=12)
40 ax1.tick_params(axis='y', labelcolor='steelblue')
41 ax1.axhline(0, color='black', linewidth=0.8)
42 ax1.set_ylim(-0.15, 0.18)
43
44 ax2 = ax1.twinx()
45 bars2 = ax2.bar(x + width/2, rf_df['Gini重要性'], width, label='随机森林Gini重要性',
    color='darkorange', edgecolor='black', linewidth=0.5, alpha=0.8)
46 ax2.set_ylabel('Gini重要性', color='darkorange', fontsize=12)
47 ax2.tick_params(axis='y', labelcolor='darkorange')
48 ax2.set_ylim(0, 0.18)
49
50 ax1.set_xticks(x)
51 ax1.set_xticklabels(order, fontsize=10)
52 ax1.set_title('九种体质对高血脂发病风险的贡献度对比', fontsize=14, fontweight='bold')
53
54 from matplotlib.patches import Patch
55 legend_elements = [

```

```

56     Patch(facecolor='steelblue', edgecolor='black', label='Logistic系数(p<0.05)'),
57     Patch(facecolor='firebrick', edgecolor='black', label='Logistic系数(p≥0.05)'),
58     Patch(facecolor='darkorange', edgecolor='black', label='随机森林Gini重要性')
59 ]
60 ax1.legend(handles=legend_elements, loc='upper_right', fontsize=9)
61
62 plt.tight_layout()
63 plt.savefig('figures/fig1b_constitution_contribution.png', dpi=300, bbox_inches='tight'
64 )
65 print("已保存figures/fig1b_constitution_contribution.png")

```

Listing 7: fig2a_roc_comparison.py: 图 2(a) ROC 对比

```

1  """
2  图2(a): 两层模型ROC曲线对比
3  (现症识别模型vs治未病预警模型)
4  """
5  import pandas as pd
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  import numpy as np
10 from sklearn.metrics import roc_curve, auc
11
12 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
13 plt.rcParams['axes.unicode_minus'] = False
14
15 df = pd.read_csv('output/p2_risk_scores.csv')
16
17 fig, ax = plt.subplots(figsize=(8, 7), dpi=300)
18
19 # 治未病预警模型ROC (使用5折交叉验证out-of-fold预测概率)
20 fpr_prev, tpr_prev, _ = roc_curve(df['disease'], df['oof_prob_prev'])
21 auc_prev = auc(fpr_prev, tpr_prev)
22 ax.plot(fpr_prev, tpr_prev, color='steelblue', linewidth=2.5,
23         label=f'治未病预警模型(AUC={auc_prev:.3f})')
24
25 # 现症识别模型ROC (使用5折交叉验证out-of-fold预测概率)
26 fpr_diag, tpr_diag, _ = roc_curve(df['disease'], df['oof_prob_diag'])
27 auc_diag = auc(fpr_diag, tpr_diag)

```

```

28 ax.plot(fpr_diag, tpr_diag, color='darkorange', linewidth=2.5,
29         label=f'现症识别模型(AUC={auc_diag:.3f})')
30
31 # 对角线
32 ax.plot([0, 1], [0, 1], 'k--', linewidth=1.5, label='随机猜测(AUC=0.500)')
33
34 ax.set_xlim([0.0, 1.0])
35 ax.set_ylim([0.0, 1.05])
36 ax.set_xlabel('假阳性率(False Positive Rate)', fontsize=12)
37 ax.set_ylabel('真阳性率(True Positive Rate)', fontsize=12)
38 ax.set_title('两层风险模型ROC曲线对比 (5折交叉验证out-of-fold)', fontsize=14,
39             fontweight='bold')
40 ax.legend(loc='lower right', fontsize=11)
41 ax.grid(True, linestyle='--', alpha=0.5)
42
43 plt.tight_layout()
44 plt.savefig('figures/fig2a_roc_comparison.png', dpi=300, bbox_inches='tight')
45 print("已保存figures/fig2a_roc_comparison.png")

```

Listing 8: fig2b_risk_score_distribution.py: 图 2(b) 风险评分分布

```

1  """
2  图2(b): 综合风险评分分布与三级风险阈值
3  """
4  import pandas as pd
5  import matplotlib
6  matplotlib.use('Agg')
7  import matplotlib.pyplot as plt
8  import numpy as np
9
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
11 plt.rcParams['axes.unicode_minus'] = False
12
13 df = pd.read_csv('output/p2_risk_scores.csv')
14
15 fig, ax = plt.subplots(figsize=(10, 6), dpi=300)
16
17 # 按风险等级分色绘制直方图
18 low = df[df['risk_level'] == '低']['risk_score']
19 mid = df[df['risk_level'] == '中']['risk_score']

```

```

20 high = df[df['risk_level'] == '高']['risk_score']
21
22 bins = np.linspace(df['risk_score'].min(), df['risk_score'].max(), 40)
23 ax.hist(low, bins=bins, alpha=0.75, label=f'低风险_{n={len(low)}}', color='#2ecc71',
24         edgecolor='white', linewidth=0.3)
25 ax.hist(mid, bins=bins, alpha=0.75, label=f'中风险_{n={len(mid)}}', color='#f39c12',
26         edgecolor='white', linewidth=0.3)
27 ax.hist(high, bins=bins, alpha=0.85, label=f'高风险_{n={len(high)}}', color='#e74c3c',
28         edgecolor='white', linewidth=0.3)
29
30 # 阈值线
31 q30 = df['risk_score'].quantile(0.30)
32 q90 = df['risk_score'].quantile(0.90)
33 anchor_high = 0.801
34
35 ax.axvline(q30, color='green', linestyle='--', linewidth=2, label=f'低中阈值_{\\theta_L}_{q30:.3f}')
36 ax.axvline(anchor_high, color='red', linestyle='--', linewidth=2, label=f'高风险阈值_{\\theta_H}_{anchor_high:.3f}')
37
38 ax.set_xlabel('综合风险评分_{R_i}', fontsize=12)
39 ax.set_ylabel('样本数', fontsize=12)
40 ax.set_title('综合风险评分分布与三级风险阈值划分', fontsize=14, fontweight='bold')
41 ax.legend(fontsize=10, loc='upper_left')
42 ax.grid(True, axis='y', linestyle='--', alpha=0.4)
43
44 plt.tight_layout()
45 plt.savefig('figures/fig2b_risk_score_distribution.png', dpi=300, bbox_inches='tight')
46 print("已保存_figures/fig2b_risk_score_distribution.png")

```

Listing 9: fig2c_weight_sensitivity.py: 图 2(c) 权重敏感性分析

```

1 """
2 图2(c): 风险评分权重敏感性分析
3 """
4 import pandas as pd
5 import matplotlib
6 matplotlib.use('Agg')
7 import matplotlib.pyplot as plt
8 import numpy as np

```

```

9
10 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
11 plt.rcParams['axes.unicode_minus'] = False
12
13 # 根据修正后代码实际运行结果 (problem2.py 敏感性分析输出)
14 scenarios = ['基准\n(组合赋权)', r'$\alpha-20\%$', r'$\alpha+20\%$', r'$\beta-20\%$', r'$\beta+20\%$']
15 high_risk_pct = [9.4, 5.1, 17.8, 9.4, 9.6]
16 changes = [0, -4.3, 8.4, 0, 0.2]
17
18 fig, ax = plt.subplots(figsize=(9, 6), dpi=300)
19
20 colors = ['steelblue'] + ['coral' if c < 0 else 'seagreen' for c in changes[1:]]
21 bars = ax.bar(scenarios, high_risk_pct, color=colors, edgecolor='black', linewidth=0.5,
22               alpha=0.85)
23
24 ax.set_ylabel('高风险人群比例(%)', fontsize=12)
25 ax.set_title('风险评分权重敏感性分析 (高风险比例变化)', fontsize=14, fontweight='bold')
26
27 ax.set_ylim(0, 22)
28 ax.axhline(9.4, color='gray', linestyle='--', linewidth=1.2, label='基准值 9.4%')
29
30 for bar, val, chg in zip(bars, high_risk_pct, changes):
31     ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
32             f'{val:.1f}%\n({chg:+.1f}pp)', ha='center', va='bottom', fontsize=9)
33
34 ax.legend(fontsize=10)
35 ax.grid(True, axis='y', linestyle='--', alpha=0.4)
36
37 plt.tight_layout()
38 plt.savefig('figures/fig2c_weight_sensitivity.png', dpi=300, bbox_inches='tight')
39 print("已保存 figures/fig2c_weight_sensitivity.png")

```

Listing 10: fig3a_intervention_trajectory.py: 图 3(a) 干预轨迹

```

1 """
2 图3(a): 样本ID_1、2、3的6个月干预轨迹
3 (痰湿积分动态变化_+调理分级标注)
4 """
5 import pandas as pd

```

```

6 import matplotlib
7 matplotlib.use('Agg')
8 import matplotlib.pyplot as plt
9 import numpy as np
10
11 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
12 plt.rcParams['axes.unicode_minus'] = False
13
14 traj1 = pd.read_csv('output/p3_id1_trajectory.csv')
15 traj2 = pd.read_csv('output/p3_id2_trajectory.csv')
16 traj3 = pd.read_csv('output/p3_id3_trajectory.csv')
17
18 fig, ax = plt.subplots(figsize=(11, 6.5), dpi=300)
19
20 month_labels = ['初始'] + [f'{m}月' for m in range(1, 7)]
21 colors = ['#e74c3c', '#3498db', '#2ecc71']
22 labels = ['ID_1(50~59岁, M=38)', 'ID_2(40~49岁, M=40)', 'ID_3(40~49岁, M=63)']
23 trajs = [traj1, traj2, traj3]
24
25 for traj, label, color in zip(trajs, labels, colors):
26     scores = [traj['月初积分'].iloc[0]] + traj['月初积分'].iloc[1:6].tolist() + [traj['
    月初积分'].iloc[-1]]
27     months = list(range(7))
28     ax.plot(months, scores, marker='o', markersize=8, linewidth=2.5, label=label, color
        =color)
29
30     # 标注调理分级变化点
31     for i in range(len(traj)-2):
32         curr_x = traj['调理分级'].iloc[i]
33         next_x = traj['调理分级'].iloc[i+1]
34         if pd.isna(curr_x) or pd.isna(next_x) or curr_x == '--' or next_x == '--':
35             continue
36         if int(curr_x) != int(next_x):
37             ax.axvline(i+1 + 0.5, color=color, linestyle='--', alpha=0.4, linewidth
                =1.5)
38             ax.annotate(f'降级→{int(next_x)}级', xy=(i+1 + 0.5, scores[i+1]),
39                 xytext=(5, 10), textcoords='offset_points',
40                 fontsize=8, color=color, fontweight='bold')
41
42 # 阈值线

```



```

43 ax.axhline(62, color='purple', linestyle=':', linewidth=1.5, alpha=0.7, label='强化调理
    阈值(62分)')
44 ax.axhline(58.5, color='brown', linestyle=':', linewidth=1.5, alpha=0.7, label='中度调
    理阈值(58.5分)')
45
46 ax.set_xlabel('干预月份', fontsize=12)
47 ax.set_ylabel('痰湿积分', fontsize=12)
48 ax.set_title('样本ID_1/2/3的6个月干预轨迹（动态逐月规划）', fontsize=14, fontweight='
    bold')
49 ax.set_xticks(range(7))
50 ax.set_xticklabels(month_labels)
51 ax.legend(fontsize=9, loc='upper_right')
52 ax.grid(True, linestyle='--', alpha=0.4)
53 ax.set_ylim(25, 70)
54
55 plt.tight_layout()
56 plt.savefig('figures/fig3a_intervention_trajectory.png', dpi=300, bbox_inches='tight')
57 print("已保存figures/fig3a_intervention_trajectory.png")

```

Listing 11: fig3b_cost_effect_scatter.py: 图 3(b) 成本-效果散点图

```

1  """
2  图3(b): 278例痰湿体质患者的成本-效果散点图
3  (按活动能力分组着色)
4  """
5  import pandas as pd
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  import numpy as np
10
11 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
12 plt.rcParams['axes.unicode_minus'] = False
13
14 df = pd.read_csv('output/p3_phlegm_patients_results.csv')
15
16 fig, ax = plt.subplots(figsize=(10, 6.5), dpi=300)
17
18 # 按活动能力分组着色
19 groups = {

```

```

20     '差(M<40)': df[df['mobility_group'] == '差(<40)'],
21     '中(40≤M<60)': df[df['mobility_group'] == '中(40-60)'],
22     '好(M≥60)': df[df['mobility_group'] == '好(≥60)']
23 }
24 colors = {'差(M<40)': '#e74c3c', '中(40≤M<60)': '#f39c12', '好(M≥60)': '#2ecc71'}
25
26 for label, subset in groups.items():
27     ax.scatter(subset['opt_cost'], subset['opt_s_drop'],
28               c=colors[label], alpha=0.6, s=40, edgecolors='black', linewidth=0.3,
29               label=f'{label}(n={len(subset)})')
30
31 ax.set_xlabel('总成本 (元)', fontsize=12)
32 ax.set_ylabel('痰湿积分下降 (分)', fontsize=12)
33 ax.set_title('278例痰湿体质患者成本-效果散点图 (按活动能力分组)', fontsize=14,
34             fontweight='bold')
35 ax.legend(fontsize=10, loc='upper_left')
36 ax.grid(True, linestyle='--', alpha=0.4)
37
38 # 添加成本上限线
39 ax.axvline(2000, color='gray', linestyle='--', linewidth=1.5, label='成本上限_2000元')
40
41 plt.tight_layout()
42 plt.savefig('figures/fig3b_cost_effect_scatter.png', dpi=300, bbox_inches='tight')
43 print("已保存figures/fig3b_cost_effect_scatter.png")

```

Listing 12: fig3c_compliance_sensitivity.py: 图 3(c) 依从性衰减敏感性分析

```

1  """
2  图3(c): 依从性衰减敏感性分析
3  """
4  import pandas as pd
5  import matplotlib
6  matplotlib.use('Agg')
7  import matplotlib.pyplot as plt
8  import numpy as np
9  from matplotlib.patches import Patch
10
11 plt.rcParams['font.sans-serif'] = ['SimHei', 'Heiti TC', 'Arial Unicode MS', 'STHeiti']
12 plt.rcParams['axes.unicode_minus'] = False
13

```

```

14 df = pd.read_csv('output/p3_sensitivity_analysis.csv')
15 summary = df.groupby('config').agg({
16     'avg_cost_278': 'first',
17     'avg_drop_278': 'first'
18 }).reindex(['保守方案', '基准方案', '乐观方案'])
19
20 fig, ax1 = plt.subplots(figsize=(9, 6.5), dpi=300)
21
22 x_pos = np.arange(len(summary))
23 width = 0.35
24
25 # 柱状图：平均成本
26 bars = ax1.bar(x_pos - width/2, summary['avg_cost_278'], width,
27               color='steelblue', edgecolor='black', linewidth=0.5, alpha=0.85,
28               label='平均成本（元）')
29 ax1.set_ylabel('平均成本（元）', color='steelblue', fontsize=12)
30 ax1.tick_params(axis='y', labelcolor='steelblue')
31 ax1.set_ylim(1200, 1400)
32
33 # 折线图：平均降分
34 ax2 = ax1.twinx()
35 line = ax2.plot(x_pos + width/2, summary['avg_drop_278'], 'o-',
36                color='darkorange', linewidth=2.5, markersize=10,
37                markeredgewidth=0.8,
38                label='平均降分（分）')
39 ax2.set_ylabel('平均降分（分）', color='darkorange', fontsize=12)
40 ax2.tick_params(axis='y', labelcolor='darkorange')
41 ax2.set_ylim(12, 20)
42
43 # 标注数值
44 for i, (cost, drop) in enumerate(zip(summary['avg_cost_278'], summary['avg_drop_278'])):
45     ax1.text(i - width/2, cost + 5, f'{cost:.1f}', ha='center', va='bottom', fontsize=9, color='steelblue')
46     ax2.text(i + width/2, drop + 0.25, f'{drop:.2f}', ha='center', va='bottom',
47             fontsize=9, color='darkorange')
48
49 ax1.set_xticks(x_pos)
50 ax1.set_xticklabels(summary.index, fontsize=11)
51 ax1.set_title('依从性衰减敏感性分析（278人平均）', fontsize=14, fontweight='bold')

```

```

51
52 from matplotlib.lines import Line2D
53 legend_elements = [Patch(facecolor='steelblue', edgecolor='black', label='平均成本
    (元)'),
54                      Line2D([0], [0], color='darkorange', marker='o', markersize=8, label
    ='平均降分 (分)')]
55 ax1.legend(handles=legend_elements, loc='upper_left', fontsize=10)
56
57 ax1.grid(True, axis='y', linestyle='--', alpha=0.4)
58
59 plt.tight_layout()
60 plt.savefig('figures/fig3c_compliance_sensitivity.png', dpi=300, bbox_inches='tight')
61 print("已保存figures/fig3c_compliance_sensitivity.png")

```

A.3 AI 使用情况说明

在本竞赛论文的撰写过程中，使用了生成式人工智能（AI）工具辅助完成以下工作：

1. **文献综述与知识检索：**利用 AI 快速检索中医体质学、Logistic 回归及 Fisher 判别分析的相关定义与公式表述，确保学术用语准确。
2. **代码框架生成：**借助 AI 生成 Python 代码的初始框架（数据读取、函数模板），随后由参赛队员根据题目约束条件自行修改参数、补全业务逻辑并调试运行。
3. **排版与格式检查：**使用 AI 辅助将文本内容转换为 L^AT_EX 格式，并依据竞赛排版规范进行页码、目录、参考文献等结构调整。

所有核心建模思路、数学推导、结果分析以及代码的最终调试均由参赛队员独立完成，AI 仅作为效率辅助工具。参赛者已对所有内容的真实性、准确性和原创性负责。